

184.686 VU Datenbanksysteme

SQL

Katja Hose

Institut für Logic and Computation

Sommersemester 2026



Informatics

- 1 Einführung
 - Overview
- 2 Datenbeschreibungssprache (DDL)
 - Basics of data definition
 - Tabellen erstellen
 - Tabellen verändern und löschen
- 3 Datenverarbeitungssprache (DML)
 - Daten einfügen
 - Daten löschen und ändern
- 4 Datenaufsichtssprache (DCL)
 - Rechteverwaltung
- 5 Ablaufsteuerungssprache (TCL)
 - Transaktionssteuerung
- 6 Anfragesprache (DQL) – SQL Grundlagen
 - Der SFW-Block
 - Einfache Anfragen
 - Mengenoperationen
 - Schachtelung von Anfragen

SQL

SQL ist eine **deklarative** Anfragesprache („was“ nicht „wie“)

SQL setzt sich aus mehreren Teilen zusammen:

- Ablaufsteuerungssprache (Transaction Control Language, TCL):
 - steuert Transaktionen
 - commit, rollback
- Datenaufsichtssprache (Data Control Language, DCL):
 - definiert/entzieht Zugriffsrechte
 - grant, revoke

Relationen vs. Tabellen

Mengen und Bags

- Bisher haben wir Relationen (Mengen) betrachtet
- In SQL betrachten wir aber Tabellen (Bags, Multimengen)

Was ist der Unterschied?

Relationen vs. Tabellen:

- Tabellen können Duplikate enthalten
- Tabellen können eine Ordnung haben
- Tabellen haben nicht unbedingt einen Schlüssel

Begriffe

- Spalten vs. Attribute
- Zeilen vs. Tupel

- 1 Einführung
- 2 Datenbeschreibungssprache (DDL)
 - Basics of data definition
 - Tabellen erstellen
 - Tabellen verändern und löschen
- 3 Datenverarbeitungssprache (DML)
- 4 Datenaufsichtssprache (DCL)
- 5 Ablaufsteuerungssprache (TCL)
- 6 Anfragesprache (DQL) – SQL Grundlagen
- 7 eSQL Tool

Grundlegende Datentypen in SQL

Datentypen

- **character(n), char(n)**
- **character varying(n), varchar(n)**
- **integer, smallint**
- **numeric(p,s), decimal(p,s), ...**
 - p – precision: max. Anzahl von Stellen (gesamt)
 - s – scale: Anzahl von Stellen nach dem Komma
- **real, double**
- **blob** oder **raw** für sehr große binäre Daten
- **clob** für große Strings
- **date** für Datumsangaben
- **xml** für XML-Dokumente
- ...

varchar(n) vs. char(n)

varchar(n) vs. char(n)

- Beide sind auf Länge n beschränkt
- **char(n)** belegt immer n Bytes
- **varchar(n)** belegt nur den benötigten Platz, plus Längeninformation

Menti

Frage 4

Menti

Frage 5

Primärschlüssel

PRIMARY KEY kennzeichnet ein Attribut als **Schlüsselattribut**.

```
CREATE TABLE professor(  
    empid      integer,  
    name       varchar(10) NOT NULL,  
    rank       char(2),  
    PRIMARY KEY(empid)  
);
```

Das Primary-Key-Constraint beinhaltet not-null- und unique-Constraints.

Menti

Frage 6

Fremdschlüssel

Fremdschlüssel können auch Schlüsselkandidaten(!) referenzieren (garantiert durch das UNIQUE-Constraint).

```
CREATE TABLE employee(  
    cpr      integer PRIMARY KEY,  
    empid    integer UNIQUE NOT NULL,  
    name     varchar(20)  
);  
  
CREATE TABLE salary(  
    cpr      integer REFERENCES employee(cpr),  
    empid    integer REFERENCES employee(empid),  
    salary   integer  
);
```

Defaultwerte

- Wird für ein Attribute beim Einfügen kein Wert angegeben, so wird dieser auf **null** gesetzt (Standard- bzw. Defaultwert).
- Beim Erstellen einer Tabelle können wir einen anderen Defaultwert definieren.

Der Defaultwert für die Farbe eines Weins soll “red” sein.

```
CREATE TABLE wine(  
    wineID    integer NOT NULL,  
    name      varchar(20) NOT NULL,  
    color     varchar(10) DEFAULT 'red',  
    year      integer,  
    vineyard  varchar(20)  
);
```

Zahlengeneratoren

Zahlengeneratoren erzeugen automatisch fortlaufende eindeutige IDs.

Beim Hinzufügen von Weinen soll automatisch eine eindeutige wineID generiert werden.

```
CREATE SEQUENCE serial START 101;
```

```
CREATE TABLE wine(  
  wineID integer PRIMARY KEY DEFAULT nextval('serial'),  
  name varchar(20) NOT NULL,  
  color varchar(10),  
  year integer,  
  vineyard varchar(20)  
);
```

Tabellen verändern

Initiale Version

```
CREATE TABLE professor(  
    empid integer NOT NULL,  
    name varchar(10) NOT NULL,  
    rank char(2)  
);
```

Attribut hinzufügen

```
ALTER TABLE professor  
    ADD COLUMN (office integer);
```

Attribut löschen

```
ALTER TABLE professor  
    DROP COLUMN name;
```

Attributtyp ändern

```
ALTER TABLE professor  
    ALTER COLUMN name type varchar(30);
```

- 1 Einführung
- 2 Datenbeschreibungssprache (DDL)
- 3 Datenverarbeitungssprache (DML)**
 - Daten einfügen
 - Daten löschen und ändern
- 4 Datenaufsichtssprache (DCL)
- 5 Ablaufsteuerungssprache (TCL)
- 6 Anfragesprache (DQL) – SQL Grundlagen
- 7 eSQL Tool
- 8 Anfragesprache (DQL) - Weiteres zu SQL

Daten einfügen

```
INSERT INTO professor VALUES  
    (2136, 'Curie', 'C4', 36),  
    (2137, 'Kant', 'C4', 7);
```

```
INSERT INTO student (studid, name) VALUES  
    (28121, 'Archimedes');
```

```
INSERT INTO student (name) VALUES  
    ('Meier');
```

```
INSERT INTO takes  
    SELECT studid, courseid  
    FROM student, course  
    WHERE title = 'Logics';
```

- 1 Einführung
- 2 Datenbeschreibungssprache (DDL)
- 3 Datenverarbeitungssprache (DML)
- 4 Datenaufsichtssprache (DCL)**
 - Rechteverwaltung
- 5 Ablaufsteuerungssprache (TCL)
- 6 Anfragesprache (DQL) – SQL Grundlagen
- 7 eSQL Tool
- 8 Anfragesprache (DQL) - Weiteres zu SQL

- 1 Einführung
- 2 Datenbeschreibungssprache (DDL)
- 3 Datenverarbeitungssprache (DML)
- 4 Datenaufsichtssprache (DCL)
- 5 Ablaufsteuerungssprache (TCL)**
 - Transaktionssteuerung
- 6 Anfragesprache (DQL) – SQL Grundlagen
- 7 eSQL Tool
- 8 Anfragesprache (DQL) - Weiteres zu SQL

6 Anfragesprache (DQL) – SQL Grundlagen

- Der SFW-Block
- Einfache Anfragen
- Mengenoperationen
- Schachtelung von Anfragen
- Allquantifizierte Anfragen

Anfragesprache

Grundgerüst einer SQL-Anfrage (SFW-Block)

SELECT <Liste von Attributen>

Projektionsliste mit arithmetischen Operationen und Aggregatfunktionen

FROM <Liste von Tabellen>

zu verwendende Tabellen, evtl. Umbenennungen

WHERE <Bedingung>;

Selektion und Join-Bedingungen, geschachtelte Anfragen

Anfragesprache

Grundgerüst einer SQL-Anfrage (SFW-Block)

```
SELECT <Liste von Attributen>  
FROM <Liste von Tabellen>  
WHERE <Bedingung>;
```

Relationale Algebra $\rightarrow$ SQL

- Projektion $\pi \rightarrow$ SELECT
- Kreuzprodukt $\times \rightarrow$ FROM
- Selektion $\sigma \rightarrow$ WHERE

Auswahl von Tabellen

Man kann Tupelvariablen für Relationen definieren.

```
SELECT *  
FROM wine AS W;
```

```
SELECT *  
FROM wine W;
```

W	wineID	name	color	year	vineyard
	1042	La Rose Grand Cru	red	1998	Château La Rose
	2168	Creek Shiraz	red	2003	Creek
	3456	Zinfandel	red	2004	Helena
	2171	Pinot Noir	red	2001	Creek
	3478	Pinot Noir	red	1999	Helena
	4711	Riesling Reserve	white	1999	Müller
	4961	Chardonnay	white	2002	Bighorn

Kreuzprodukt (Kartesisches Produkt)

Bei mehr als einer Tabelle in der FROM-Klausel wird das Kreuzprodukt gebildet.

```
SELECT *  
FROM wine, producer;
```

Als Ergebnis werden **alle** Kombinationen generiert!

Tupelvariablen für mehrfachen Zugriff

Die Definition von Tupelvariablen erlaubt mehrfachen Zugriff auf eine Tabelle (z.B. Self-Join).

```
SELECT *  
FROM wine w1, wine w2;
```

Die Spalten des Ergebnisses lauten:

w1.wineID, w1.name, w1.color, w1.year, w1.vineyard,
w2.wineID, w2.name, w2.color, w2.year, w2.vineyard

Natural Join

Frühe SQL-Versionen

- Kennen nur Kreuzprodukt, keinen expliziten Join-Operator
- Joinbedingung durch Prädikat in der WHERE-Klausel definiert

Example

```
SELECT *  
FROM wine, producer  
WHERE wine.vineyard = producer.vineyard;
```

Natural Join als expliziter Operator

Natural Join

```
SELECT *  
FROM wine NATURAL JOIN producer;
```

Was sind jetzt die Spalten des Ergebnisses? Was ist das Ergebnis?

wine (wineID, name, color, year, ~~vineyard~~)
producer (vineyard, area, region)

- Ergebnisspalten: wineID, name, color, year, vineyard, area, region
- Ergebnis: Kreuzprodukt

Menti

Frage 10

Kreuzprodukt als expliziter Operator

Kreuzprodukt (Kartesisches Produkt)

```
SELECT *  
FROM wine, producer;
```

als **CROSS JOIN**

```
SELECT *  
FROM wine CROSS JOIN producer;
```

Menti

Frage 11

Tupelvariable

```
SELECT name, year, vineyard  
FROM wine, producer  
WHERE wine.vineyard = producer.vineyard;
```

Ist dies eine gültige SQL-Anfrage?

Die Referenz ist nicht eindeutig:
Das Resultat des Joins hat zwei Spalten mit dem Namen vineyard.

Richtiger Ausdruck

```
SELECT name, year, producer.vineyard  
FROM wine, producer  
WHERE wine.vineyard = producer.vineyard;
```

Die SELECT-Klausel

Festlegung der Projektionsattribute

```
SELECT [DISTINCT] projektionsliste  
FROM ...
```

Projektionsliste

- Liste von Spalten
- Spezialfall: „*****“ steht für alle Spalten der Tabellen in der From-Klausel
- Arithmetische Ausdrücke bestehend aus Konstanten und Spalten der Tabellen
- Aggregatfunktionen über Spalten der Tabellen (mehr dazu später)

Duplikateliminierung

```
SELECT name  
FROM wine;
```

Liefert eine Ergebnisrelation als Multimenge (Bag)

name
La Rose Grand Cru
Creek Shiraz
Zinfandel
Pinot Noir
Pinot Noir
Riesling Reserve
Chardonnay

Duplikateliminierung

```
SELECT DISTINCT name  
FROM wine;
```

Entspricht der Projektion der Relationenalgebra

name
La Rose Grand Cru
Creek Shiraz
Zinfandel
Pinot Noir
Riesling Reserve
Chardonnay

Sortierung

```
SELECT      empid, name, rank  
FROM        professor  
ORDER BY    rank DESC, name ASC;
```

Reihenfolge:

- ASC: aufsteigend (ascending)
- DESC: absteigend (descending)

Default: ASC

Die WHERE-Klausel

```
SELECT ...  
FROM ...  
WHERE condition;
```

condition beschreibt Bedingungen, die für alle Ergebnistupel gelten müssen

- Vergleiche von Attributen mit anderen Attributen und/oder Konstanten:
Vergleichsoperatoren: =, <>, >, <, ≥, ≤, LIKE, BETWEEN, IN, IS
- Bedingungen in der WHERE-Klausel können logisch kombiniert werden mit
AND, OR, NOT

Jede Bedingung, die für einen Theta-Join definiert werden kann, kann in der WHERE-Klausel verwendet werden.

“IS NULL” wird verwendet um auf null-Werte zu testen!

Anfrage mit einer Tabelle

```
SELECT empid, name  
FROM professor  
WHERE rank = 'C4';
```

professor	
empid	name
2125	Socrates
2126	Russel
2136	Curie
2137	Kant

professor			
empid	name	rank	office
2125	Socrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7

SQL legt nicht fest, in welcher Reihenfolge Selektion, Projektion und Join ausgeführt werden.

Anfrage mit mehreren Tabellen

professor (empid, name, rank, office)

course (courseid, title, ects, taughtby)

Welcher Professor liest „Bioethik“ (SQL und relationale Algebra)?

```
SELECT name, title
```

```
FROM professor, course
```

```
WHERE empid = taughtby AND title = 'Bioethics';
```

Ausdruck in relationaler Algebra

$$\pi_{name, title}(\sigma_{empid=taughtby \wedge title='Bioethics'}(professor \times course))$$

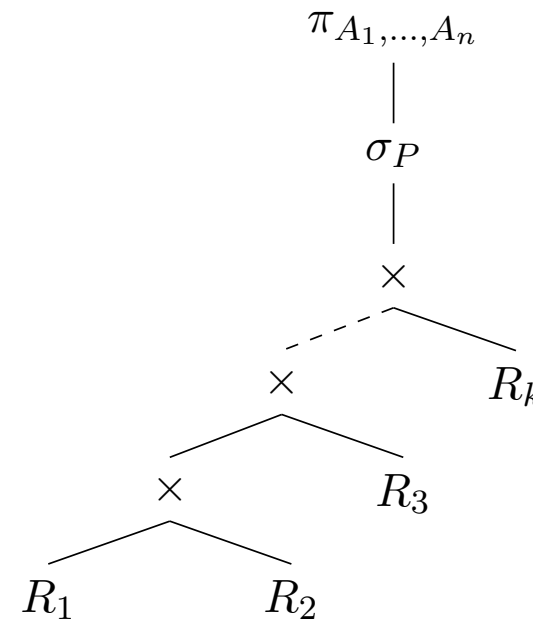
Übersetzung von SQL-Anfragen in relationale Algebra

Allgemeine Form einer
(ungeschachtelten) SQL-Anfrage

```
SELECT  $A_1, \dots, A_n$   
FROM  $R_1, \dots, R_k$   
WHERE  $P$ ;
```

Übersetzung in relationale Algebra

$$\pi_{A_1, \dots, A_n}(\sigma_P(R_1 \times \dots \times R_k))$$



Mengenoperationen

Mengenoperationen erfordern Vereinigungskompatibilität:
Gleiche Anzahl von Attributen mit kompatiblen Domänen.

- beide Wertebereiche sind gleich
- beide sind auf character basierende Wertebereiche (unabhängig von der Länge der Strings)
- beide sind numerische Wertebereiche (unabhängig vom genauen Typ), z.B. integer oder float

Ergebnisschema: Spaltennamen der **ersten** Tabelle

```
SELECT A, B, C FROM R1  
UNION  
SELECT A, C, D FROM R2;
```


Duplikate und Mengenoperationen

Bei Mengenoperationen ist Duplikateliminierung (distinct) default!

```
(SELECT name  
FROM assistant)
```

```
UNION
```

```
(SELECT name  
FROM assistant);
```

```
(SELECT name  
FROM assistant)  
UNION DISTINCT
```

```
(SELECT name  
FROM assistant);
```

```
(SELECT name  
FROM assistant)
```

```
UNION ALL
```

```
(SELECT name  
FROM assistant);
```

Durchschnitt (INTERSECT) und Mengendifferenz (EXCEPT) werden auch unterstützt.

Schachtelung von Anfragen

Für Vergleiche mit Wertemengen sind Unteranfragen notwendig.

- Standardvergleiche in Verbindung mit den Quantoren ALL ($\forall$) und ANY ($\exists$)
- Spezielle Prädikate für den Zugriff auf Mengen: IN und EXISTS

Unkorrelierte Unteranfragen

Notation

attribut IN (SFW-Block)

```
SELECT name  
FROM professor  
WHERE empid IN (SELECT taughtby  
                FROM course);
```

Vergleich von Wert mit Menge von Werten

Unkorrelierte Unteranfragen

Welche Weine wurden in der Region “Bordeaux” produziert (mit Hilfe von IN).

wine (wineID, name, color, year, vineyard)

producer (vineyard, area, region)

```
SELECT name
FROM wine
WHERE vineyard IN (SELECT vineyard
                   FROM producer
                   WHERE region='Bordeaux');
```

Können wir die gleiche Anfrage formulieren, ohne IN zu verwenden?

```
SELECT name
FROM wine NATURAL JOIN producer
WHERE region = 'Bordeaux';
```

Negation des IN-Prädikats

Simulation des Differenzoperators

$$\pi_{\text{vineyard}}(\text{producer}) - \pi_{\text{vineyard}}(\text{wine})$$

durch folgende SQL-Anfrage

```
SELECT vineyard
FROM producer
WHERE vineyard NOT IN (
    SELECT vineyard FROM wine);
```

Korrelierte Unteranfragen

```
SELECT name  
FROM professor p  
WHERE EXISTS (SELECT 42  
              FROM course v  
              WHERE v.taughtby = p.empid);
```

Was passiert, wenn wir das **SELECT *** in der Unteranfrage ändern?

Erhalten wir nun andere Ergebnisse?

Nein, wir testen nur auf Existenz.

Ist das Ergebnis der inneren Anfrage leer?

Für jedes Ergebnistupel in der inneren Anfrage wird 42 zurückgegeben.

Quantoren: IN vs. EXISTS

IN

Ist das „linke Tupel“ in der „rechten Menge“ enthalten?

Beispiel: ... (studid, courseid) **IN** (SELECT * FROM takes);

EXISTS

Ist die „rechte Seite“ leer?

Beispiel: ... **EXISTS** (SELECT * FROM takes WHERE ...);

Quantor: ANY

Werte mit einer Menge von Werten vergleichen

```
SELECT name, semester  
FROM student  
WHERE semester >= ANY (SELECT semester  
                           FROM student);
```


Quantor: ALL

Werte mit einer Menge von Werten vergleichen

```
SELECT name  
FROM student  
WHERE semester >= ALL (SELECT semester  
                        FROM student);
```

Was passiert, wenn „>=“ durch „>“ ersetzt wird? Empty result

Quantoren: ALL vs. ANY

wine (wineID, name, color, year, vineyard)

producer (vineyard, area, region)

Finde den ältesten Wein (verwende ALL).

```
SELECT *  
FROM wine  
WHERE year <= ALL (  
    SELECT year FROM wine);
```

Finde alle Weingüter, die Rotweine herstellen (verwende ANY).

```
SELECT *  
FROM producer  
WHERE vineyard = ANY (  
    SELECT vineyard FROM wine  
    WHERE color = 'red');
```

Unteranfragen in der WHERE-Klausel ohne Quantoren

```
SELECT *  
FROM grades  
WHERE grade < (SELECT AVG(grade)  
               FROM grades);
```

Vergleich eines Wertes mit einem anderen Wert

Unteranfragen in der SELECT-Klausel

- Die Unteranfrage wird für jedes Lösungstupel evaluiert.
- Die Unteranfrage ist korreliert.

```
SELECT empid, name, (SELECT SUM(ects)
                     FROM course
                     WHERE taughtby = empid) AS teachingLoad
FROM professor;
```

Berechne einen Wert für jedes äußere Tupel.

COALESCE()

Geben Sie eine Liste **aller** Studierenden aus inkl. deren Noten für abgeschlossene Kurse. Für Studierende, die keinen Kurs besucht haben, sollen courseID und grade den Wert 0 annehmen.

```
SELECT s.studID, s.name,  
       COALESCE(g.courseID,0),  
       COALESCE(g.grade,0)  
FROM student s LEFT OUTER JOIN grades  
                ON s.studID = g.studID;
```

Achtung: Typen müssen kompatibel sein

Allquantifizierte Anfragen

Welche Studierenden haben **alle** 4-ECTS Lehrveranstaltungen gehört?

student(studID, name, semester)

takes(studID, courseID)

course(courseID, title, ects, taughtBy)

$$\text{takes} \div \pi_{\text{courseID}}(\sigma_{\text{ects}=4}(\text{course}))$$

SQL stellt **keinen Allquantor** zur Verfügung.

Allquantifizierte Anfragen – Nicht direkt in SQL

Welche Studierende haben **alle** 4-ECTS Lehrveranstaltungen gehört?

SQL stellt keinen Allquantor zur Verfügung.

Realisierung durch Logische Äquivalenz (mittels $2 \times \text{NOT EXISTS}$).

Allquantifizierte Anfragen – “Logische Umformung”

Umformulierung in äquivalente Anfrage mit Negation und Existenzquantor (Prädikatenlogik)

$$\forall x F(x) \Leftrightarrow \neg \exists x (\neg F(x))$$

Welche Studierenden haben **alle** 4-ECTS Lehrveranstaltungen gehört?

- $\Leftrightarrow$ Suchen Sie jene Studierenden, für die **gilt**:
sie haben **alle** 4-ECTS Lehrveranstaltungen **gehört**
- $\Leftrightarrow$ Suchen Sie jene Studierenden, für die **nicht gilt**:
es **gibt eine** 4-ECTS Lehrveranstaltung,
die der/die Studierende **nicht gehört** hat.

Allquantifizierte Anfragen – “Logische Umformung”

SQL-Umsetzung folgt nun direkt aus:

Suchen Sie jene Studierende für die **nicht gilt**:

es gibt eine 4-ECTS Lehrveranstaltung, für die **nicht gilt**:
die/der Studierende hat diese Lehrveranstaltung gehört.

```
SELECT s.*  
FROM student s  
WHERE NOT EXISTS (SELECT *  
                    FROM course c  
                    WHERE c.ects = 4 AND  
                          NOT EXISTS (SELECT *  
                                      FROM takes t  
                                      WHERE t.courseID = c.courseID  
                                      AND s.studID = t.studID ));
```

Allquantifizierte Anfragen – “Logische Umformung”

SQL-Umsetzung folgt nun direkt aus:

Suchen Sie jene Studierende für die **nicht gilt**:

es gibt eine 4-ECTS Lehrveranstaltung, für die **nicht gilt**:
die/der Studierende hat diese Lehrveranstaltung gehört.

```
SELECT s.*  
FROM student s  
WHERE NOT EXISTS (SELECT *  
                    FROM course c  
                    WHERE c.ects = 4 AND  
                          s.studID NOT IN (SELECT t.studID  
                                           FROM takes t  
                                           WHERE t.courseID = c.courseID ));
```

7 eSQL Tool

▼ SQL  Für Teilnehmer/innen verborgen

 SQL Dump (university)

...

eSQL Tool



Link to eSQL Tool

 Für Teilnehmer/innen verborgen



Kurze Beschreibung des eSQL Tools

 Für Teilnehmer/innen verborgen



Description of the eSQL Tool

 Für Teilnehmer/innen verborgen

 Für Teilnehmer/innen verborgen

Übungsblatt 5 & 6 / Exercise Sheet 5 & 6



Übungsblatt 5

Abgabebeginn: Montag, 22. April 2024, 05:00

Abgabeende: Freitag, 28. Juni 2024, 23:00

Verbindung zu externem Server: OK



SignOn

11702806 dbai
Timo Merkl Database and Artificial
Intelligence Group

Authentifizierung erfolgreich

[Klicken Sie hier, um das eSQL-Übungssystem in einem neuen Fenster zu öffnen.](#)

Sollten Sie die Fehlermeldung "Nicht angemeldet!" erhalten, so laden Sie bitte die eSQL-Aktivität in TUWEL erneut.

[Kontakt & Impressum](#)

Datenbanksysteme SQL-Übungssystem

Allgemeines

Mit dem Übungssystem können Sie SQL Aufgabestellungen interaktiv lösen. Das Tool liefert Ihnen Feedback zu den von Ihnen eingegebenen Lösungen. Auf der einen Seite unterstützt es damit das Finden von Fehlern in Ihrer Query, auf der anderen Seite bietet es die Möglichkeit, Queries zu optimieren, welche zwar bereits die richtige Lösungsmenge liefern, allerdings vom erwarteten Lösungsweg abweichen. In diesem Fall wird Ihnen ein Warning angezeigt. Wenn Sie der Meinung sind, Ihre Lösung ist korrekt und kann nicht mehr optimiert werden, dann können Sie das auch ohne Weiters so stehen lassen. (Es gibt bei vielen Beispielen eine Reihe von richtigen Lösungen. Leider kann nicht immer jede korrekte Lösung als solche erkannt werden.)

Das Übungssystem

- Die aktuell für die Übung relevante Datenbankdefinition und die Übungsbeispiele finden Sie unter "**Aktueller Kurs**". Mit einem Klick auf "Übung" gelangen Sie direkt zur Beschreibung, der Datenbankübersicht und den Beispielaufgaben.
- Lösungen werden bis zur Abgabe-Deadline im System gespeichert, danach gehen Änderungen nach dem Logout verloren. Alte Kurse sind ebenfalls verfügbar, um Ihnen die Möglichkeit zum weiteren Üben zu bieten. Allerdings werden hier Ihre Eingaben nicht gespeichert.
- Sie können unter dem Menüpunkt "Aufgaben" Ihre Aufgabe abgeben. Danach werden Änderungen nicht mehr gespeichert.
- Mittels Button "Prf-Demo" können sie den eSQL-Test simulieren. (Hinweis: Beim Test werden andere Datenbanken, bestehend aus ca fünf Entitäten, verwendet.)
- Ihre Eingaben zu alten Kursen und der "Prf-Demo" fließen nicht in die Benotung ein!

Unter "Hilfe" finden Sie eine genauere Erklärung zu den Funktionen des eSQL-Tools.

■ Aktueller Kurs: Verlag - Die Datenbank

Sie können Ihre Lösungen zu diesem Kurs bis 28.06.2024 23:00 bearbeiten. Änderungen werden gespeichert.

Ein Verlag benoetigt eine Datenbank.

A publisher wants a database.

Alte Kurse

■ Baederverwaltung - Die Datenbank

Eine Baederverwaltung benoetigt eine Datenbank.

■ BetaRelease - Die Datenbank

Eine Softwarefirma benoetigt eine Datenbank.

Verlag - Die Datenbank

Ein Verlag benoetigt eine Datenbank.

A publisher wants a database.

[Druckansicht](#)**Beschreibung**

Die Geschäftsführung des Literaturverlags Leserratte möchte eine Datenbank anlegen, um die wichtigsten Informationen über die Veröffentlichungen des Verlags zu speichern.

In der Datenbank soll gespeichert werden, welche Personen an den Veröffentlichungen mitarbeiten. Personen werden eindeutig durch ihren Namen (NAME) identifiziert. Ausserdem wird zu jeder Person das Geburtsdatum (GEBDAT) und eine Kurzbiographie für die Website des Verlags (KURZBIO) gespeichert.

Bei den Personen kann es sich um Autor/innen, Übersetzer/innen oder Lektor/innen handeln. Eine Person kann auch Autor/in und Übersetzer/in zugleich sein.

Bei Lektor/innen wird in der Datenbank erfasst, welche Sprachen sie beherrschen. Jede Sprache wird dabei durch einen eindeutigen Namen (NAME) identifiziert. Ein Lektor oder eine Lektorin kann beliebig viele Sprachen beherrschen. Bei Übersetzer/innen muss gespeichert werden, in welche Sprachen sie übersetzen können und aus welchen Originalsprachen sie übersetzen können. Diese beiden Beziehungen sind voneinander unabhängig -- es kann sein, dass jemand z.B. englische Texte ins Deutsche übersetzen kann, aber keine deutschen Texte ins Englische.

Für jedes Werk, das im Verlag Leserratte erscheint, wird gespeichert, welche Autor/innen es geschrieben haben. Dabei ist zu beachten, dass auch mehrere Autor/innen gemeinsam ein Werk schreiben können. Werke werden durch ihren Titel (TITEL) und das Jahr, in dem sie geschrieben wurden (JAHR) identifiziert. Zu jedem Werk soll ausserdem noch eine kurze Beschreibung für die Verlagswebsite (BESCHREIBUNG) gespeichert werden.

Ein Werk kann eine Übersetzung eines anderen Werks sein. In der Datenbank soll gespeichert werden, bei welchen Werken es sich um Übersetzungen handelt und wer sie übersetzt hat. Der Einfachheit halber nehmen wir an, dass es nicht möglich ist, dass mehrere Personen gemeinsam eine Übersetzung eines Werks erstellen. Von einem Werk kann es aber mehrere Übersetzungen geben, die auch von unterschiedlichen Übersetzer/innen sein können.

Zu den Werken wird ausserdem noch gespeichert, zu welchen Genres (Science-Fiction-Roman, Gedichtband ...) sie gehören. Jedes Werk wird einem oder mehreren Genres zugeordnet. Jedes Genre wird durch eine ID (GID) eindeutig identifiziert und hat ausserdem noch einen Namen (NAME) und eine kurze Beschreibung (BESCHREIBUNG). Ein Genre kann (muss aber nicht) ein Subgenre von einem oder mehreren anderen Genres sein.

Von jedem Werk, das im Verlag Leserratte erscheint, gibt es eine oder mehrere Auflagen. Eine Auflage wird durch das Werk und durch eine zusätzliche Auflagennummer (NUMMER) eindeutig identifiziert. Die Auflagennummer allein ist im Allgemeinen nicht eindeutig. Ausserdem soll das Jahr gespeichert werden, in dem die Auflage erscheint (JAHR), und die Lektor/innen, die an der Auflage mitgearbeitet haben. An jeder Auflage arbeitet mindestens ein Lektor/eine Lektorin mit.

Eine kurze Beschreibung und ein ER-Diagramm.

Relationenmodell

arbeitet_in	(sprache, name)
auflage	(titel, jahr, nummer, aufljahr)
autorin	(name)
gehört_zu	(titel, jahr, gid)
genre	(gid, name, beschreibung)
lektoriert	(name, titel, jahr, nummer)
lektorin	(name)
person	(name, gebdat, kurzbio)
publikation	(titel, jahr, nummer, pid, seiten)
schreibt	(person, titel, jahr)
sprache	(name)
subgenre	(untergeordnet, uebergeordnet)
uebersetzerin	(name)
uebersetzt_aus	(sprache, name)
uebersetzt_in	(sprache, name)
uebersetzung	(original_titel, original_jahr, uebersetzung_titel, uebersetzung_jahr, uebersetzerin)
vertrag	(vid, datum, beschreibung, mit)
werk	(titel, jahr, beschreibung)

Datenbankabfrage

Daten anzeigen: [Tabelle] ▼

Switch editor

ausführen

Keine Abfrage eingegeben.

Hier können Anfragen getestet werden und das Datenbankschema wird erklärt.

- #1  Geben Sie Titel und Beschreibung aller Werke aus, die im Jahr 2004 oder später geschrieben wurden. Die Ergebnisse sollen aufsteigend nach dem Titel sortiert sein. Tipp: Verwenden Sie die Funktion TO_DATE mit dem Datumsformat 'YYYY'.
- English:
List the title ("Titel") and description ("Beschreibung") of all opus/corpus ("Werke") that were written ("geschrieben") in the year 2004 or later. The results shall be ordered ascending by the title ("Titel"). Hint: Use the function TO_DATE and the date format 'YYYY'.
-
- #2  Gesucht sind Name und Geburtsdatum aller Uebersetzer/innen, die aus dem Russischen uebersetzen koennen und an mindestens zwei Uebersetzungen mitgearbeitet haben, absteigend sortiert nach dem Geburtsdatum. Verwenden Sie zur Ausgabe der Geburtsdaten die Funktion TO_CHAR und das Datumsformat 'DD-MM-YYYY'.
- English:
List the names ("Name") and dates of birth ("Geburtsdatum") of all translators ("Uebersetzer/innen"), who can translate ("uebersetzen") from ("aus") Russian and have worked on at least two translations, ordered descending by the date of birth. Hint: To display the dates of birth, use the function TO_CHAR and the date format 'DD-MM-YYYY'.
-
- #3.1  Geben Sie die Titel und Beschreibungen aller Werke aus, die keine Uebersetzungen sind. Die Ergebnisse sollen aufsteigend nach dem Titel sortiert werden.
- English:
List the title ("Titel") and description ("Beschreibung") of all opus/corpus ("Werke") that are no translations ("Uebersetzungen"). The results must be ordered ascending by the title.
- #3.2  Geben Sie die Titel und Beschreibungen aller Werke aus, die keine Uebersetzungen sind und nicht vor 2004 geschrieben wurden. Die Ergebnisse sollen aufsteigend nach dem Titel sortiert werden.
- English:
List the title ("Titel") and description ("Beschreibung") of all opus/corpus ("Werke") that are no translations ("Uebersetzungen") and that were not written before the year 2004. The results must be ordered ascending by the title. Hint: Use the function TO_DATE and the date format 'YYYY'.
- #3.3  Geben Sie die Titel und Beschreibungen aller Werke aus, die keine Uebersetzungen sind, nicht zum Genre "Science Fiction" gehoeren und nicht vor 2004 geschrieben wurden. Die Ergebnisse sollen aufsteigend nach dem Titel sortiert werden.
- English:
List the title ("Titel") and description ("Beschreibung") of all opus/corpus ("Werke") that are no translations ("Uebersetzungen"), do not belong to the genre "Science Fiction", and were not written before the year 2004. The results must be ordered ascending by the title. Hint: Use the function TO_DATE and the date format 'YYYY'.
-
- #4.1  Gesucht sind Auflagennummer und Jahreszahl aller Auflagen des Werks "Der Mann ohne Eigenschaften", aufsteigend sortiert nach der Auflagennummer. Verwenden Sie zur Ausgabe der Jahreszahl die Funktion TO_CHAR mit dem Datumsformat 'YYYY'.

Hier sieht man die verschiedenen Aufgaben (3.1 + 3.2 + 3.3 ist eine Aufgabe).

■ **Aufgabe #1**

Geben Sie Titel und Beschreibung aller Werke aus, die im Jahr 2004 oder später geschrieben wurden. Die Ergebnisse sollen aufsteigend nach dem Titel sortiert sein. Tipp: Verwenden Sie die Funktion TO_DATE mit dem Datumsformat 'YYYY'.

English:

List the title ("Titel") and description ("Beschreibung") of all opus/corpus ("Werke") that were written ("geschrieben") in the year 2004 or later. The results shall be ordered ascending by the title ("Titel"). Hint: Use the function TO_DATE and the date format 'YYYY'.

■ **Eingabe**

Switch editor



ausführen

Keine Abfrage eingegeben.

Hier kann die SQL Abfrage eingegeben werden

Das sind die einzelnen Aufgaben.

Kurse **Verlag - Die Datenbank**

[Übersicht](#) [Datenbank](#) [Aufgaben](#)  2 3 3 3 4 4 4 5 6 6 7 8 9 9 10

Aufgabe #1

Geben Sie Titel und Beschreibung aller Werke aus, die im Jahr 2004 oder später geschrieben wurden. Die Ergebnisse sollen aufsteigend nach dem Titel sortiert sein. Tipp: Verwenden Sie die Funktion TO_DATE mit dem Datumsformat 'YYYY'.

English:

List the title ("Titel") and description ("Beschreibung") of all opus/corpus ("Werke") that were written ("geschrieben") in the year 2004 or later. The results shall be ordered ascending by the title ("Titel"). Hint: Use the function TO_DATE and the date format 'YYYY'.

Eingabe

[Switch editor](#)

```
select *  
from werk
```



[ausführen](#)



- ✓ **Struktur OK** *Die strukturellen Eigenschaften der Eingabe sind in Ordnung*
- ✓ **Tabellen OK** *Die verwendeten Tabellen in der Eingabe sind in Ordnung*
- ✗ **Fehlende Konstanten** *Es fehlen in der Eingabe Konstanten, die erwartet werden [2004; YYYY]*
- ✗ **zu viele Zeilen (+)** *Das Ergebnis beinhaltet Zeilen, die nicht erwartet werden*
- ✗ **zu viele Spalten (+)** *Das Ergebnis beinhaltet Spalten, die nicht erwartet werden*
- ✗ **Sortierung falsch** *Das Ergebnis ist nicht wie erwartet sortiert*

Ergebnis
Erwartet
Differenz

titel	beschreibung	jahr
Winterreise	Ausgehend von Franz Schuberts berühmtem Liederzyklus, durchwandert das Ich in Elfriede Jelineks Winterreise den Wahrwitz unserer Gegenwart.	2011-01-01 00:00:00.0
Das Wetter vor 15 Jahren	Einer der seltsamsten und intelligentesten Liebesromane der letzten Jahre. Ausgezeichnet mit dem Wilhelm-Raabe-Literaturpreis.	2006-01-01 00:00:00.0
Ewig Dein	Daniel Glattauers neuester Liebesroman	2012-01-01 00:00:00.0
Wir schlafen nicht	Kathrin Roeggglas Stimmencollage wir schlafen nicht berichtet aus der Arbeitswelt der Online- und Beratergesellschaft.	2004-01-01 00:00:00.0
Infinite Jest	Einer der bekanntesten amerikanischen postmodernen Romane	1996-01-01 00:00:00.0
Oblivion	Neuer Erzählband von David Foster Wallace	2004-01-01 00:00:00.0
Unendlicher Spass	Einer der bekanntesten amerikanischen postmodernen Romane	2009-01-01 00:00:00.0
Vergessenheit	Neuer Erzählband von David Foster Wallace	2008-01-01 00:00:00.0
23000	Dritter Teil von Sorokins Eis-Trilogie.	2009-01-01 00:00:00.0
23000	Dritter Teil von Sorokins Eis-Trilogie.	2010-01-01 00:00:00.0
Der Zuckerkreml	Erzählband	2008-01-01 00:00:00.0
Der Zuckerkreml	Erzählband	2010-01-01 00:00:00.0
Der Schneesturm	Was beginnt wie eine Erzählung aus dem 19. Jahrhundert, entpuppt sich als fantastische Irrfahrt durch das laendliche Russland einer nahen Zukunft.	2010-01-01 00:00:00.0
Der Schneesturm	Was beginnt wie eine Erzählung aus dem 19. Jahrhundert, entpuppt sich als fantastische Irrfahrt durch das laendliche Russland einer nahen Zukunft.	2012-01-01 00:00:00.0

Das Symbol ist relevant für die Bewertung.

Symbole

Lösen von Beispielen

Wenn Sie ein Beispiel lösen (korrekt oder nicht) wird im Feedback-Bereich Rückmeldung vom System angezeigt. Dabei wird eine Liste von Informationen angezeigt, sowie ein Icon, das Aufschluß über den Status Ihrer Lösung gibt. Die möglichen Zustände Ihrer Lösung sind wie folgt:

-  **DEFAULT**
Dieser Status wird angezeigt, wenn keine Eingabe gemacht wurde, oder keiner der anderen Zustände zutrifft. Am Icon wird die Beispielnummer angezeigt.
-  **ERROR**
Dieser Status wird angezeigt, wenn Ihre Lösung aufgrund eines Fehlers nicht ausgeführt werden kann.
-  **INCORRECT**
Dieser Status wird angezeigt, wenn die Lösung zwar ausführbar ist, aber das Ergebnis nicht korrekt ist.
-  **INCOMPARABLE**
Das ist ein spezieller Status, der angezeigt wird, wenn das Ergebnis Ihrer Lösung mit dem erwarteten Ergebnis nicht verglichen werden kann, weil sich die Ergebnisse zu stark voneinander unterscheiden. In diesem Fall wird auch keine Ergebnistabelle dargestellt!
-  **CORRECT**
Dieser Status wird angezeigt, wenn Ihre Lösung korrekt ist!
-  **WARNING**
Dieser Status wird angezeigt, wenn Ihre Lösung zwar das exakt richtige Ergebnis liefert, aber Ihr Lösungsweg nicht erwartet wird. Es gibt in vielen Beispielen eine Reihe von richtigen Lösungen. Leider kann nicht immer jede korrekte Lösung als solche erkannt werden. Dieser Status gibt an, dass Ihre Lösung möglicherweise nicht dem geforderten Lösungsweg entspricht.
Wenn Sie der Meinung sind, Ihre Lösung ist korrekt, dann können Sie das so stehen lassen.
Als Hinweis sei gesagt: Die Lösungen werden bei der Prüfung für die Beurteilung anhand anderer Daten getestet. Nur wenn dort ebenfalls das erwartete Ergebnis erzielt wird, wird die Lösung als korrekt bewertet.

Aufgabe nicht gelöst / Anfrage falsch

- Default: Aufgabe wurde noch nicht probiert.
- Error: Anfrage konnte nicht ausgeführt werden.
- Incorrect: Anfrage liefert falsches Ergebnis.
- Incomparable: Anfrage liefert Ergebnis, das nicht mit dem erwarteten verglichen werden kann.

Symbole

Lösen von Beispielen

Wenn Sie ein Beispiel lösen (korrekt oder nicht) wird im Feedback-Bereich Rückmeldung vom System angezeigt. Dabei wird eine Liste von Informationen angezeigt, sowie ein Icon, das Aufschluß über den Status Ihrer Lösung gibt. Die möglichen Zustände Ihrer Lösung sind wie folgt:

-  **DEFAULT**
Dieser Status wird angezeigt, wenn keine Eingabe gemacht wurde, oder keiner der anderen Zustände zutrifft. Am Icon wird die Beispielnummer angezeigt.
-  **ERROR**
Dieser Status wird angezeigt, wenn Ihre Lösung aufgrund eines Fehlers nicht ausgeführt werden kann.
-  **INCORRECT**
Dieser Status wird angezeigt, wenn die Lösung zwar ausführbar ist, aber das Ergebnis nicht korrekt ist.
-  **INCOMPARABLE**
Das ist ein spezieller Status, der angezeigt wird, wenn das Ergebnis Ihrer Lösung mit dem erwarteten Ergebnis nicht verglichen werden kann, weil sich die Ergebnisse zu stark voneinander unterscheiden. In diesem Fall wird auch keine Ergebnistabelle dargestellt!
-  **CORRECT**
Dieser Status wird angezeigt, wenn Ihre Lösung korrekt ist!
-  **WARNING**
Dieser Status wird angezeigt, wenn Ihre Lösung zwar das exakt richtige Ergebnis liefert, aber Ihr Lösungsweg nicht erwartet wird. Es gibt in vielen Beispielen eine Reihe von richtigen Lösungen. Leider kann nicht immer jede korrekte Lösung als solche erkannt werden. Dieser Status gibt an, dass Ihre Lösung möglicherweise nicht dem geforderten Lösungsweg entspricht.
Wenn Sie der Meinung sind, Ihre Lösung ist korrekt, dann können Sie das so stehen lassen.
Als Hinweis sei gesagt: Die Lösungen werden bei der Prüfung für die Beurteilung anhand anderer Daten getestet. Nur wenn dort ebenfalls das erwartete Ergebnis erzielt wird, wird die Lösung als korrekt bewertet.

Anfrage sicher richtig

- Correct: Aufgabe wird vom System als richtig erkannt.

8 Anfragesprache (DQL) - Weiteres zu SQL

- Weitere Join-Varianten
- Aggregation
- Gruppierung
- Null-Werte
- Rekursion
- Anzahl von Ergebnissen begrenzen

Weiteres zu SQL

Erweiterungen des SFW-Blocks

- FROM-Klausel: Weitere Join-Varianten
- WHERE-Klausel: Weiter Arten von Quantoren und Constraints
- SELECT-Klausel: Verwendung von skalaren Operationen und Aggregatfunktionen

- Join-Varianten
- GROUP BY und HAVING
- CASE,...
- Window Funktionen
- WITH
- Rekursive Anfragen
- ...

Joins

Join variants

- CROSS JOIN
- NATURAL JOIN
- JOIN or **INNER JOIN**
- **LEFT**, **RIGHT**, or **FULL OUTER JOIN**

Standard Formulierung

SELECT *

FROM R_1, R_2

WHERE $R_1.A = R_2.B$;

Alternative Formulierung

SELECT *

FROM R_1 JOIN R_2 ON $R_1.A = R_2.B$;

8 Anfragesprache (DQL) - Weiteres zu SQL

- Weitere Join-Varianten
- **Aggregation**
- Gruppierung
- Null-Werte
- Rekursion
- Anzahl von Ergebnissen begrenzen

Aggregatfunktionen

Wie können wir die folgende Anfrage in SQL formulieren?

- Den durchschnittlichen Preis aller Artikel im Angebot
- Gesamtumsatz aller verkauften Produkte

Aggregatfunktionen berechnen neue Werte für eine Spalte.

Aggregatfunktionen

AVG, MAX, MIN, COUNT, SUM

Aggregatfunktionen und Duplikate

Der Argumentspalte (außer im Falle COUNT(*)) kann optional ein DISTINCT oder ein ALL hinzugefügt werden.

- DISTINCT: bevor die Aggregatfunktionen ausgewertet wird, werden Duplikate entfernt
- ALL: Duplikate werden für die Auswertung herangezogen (**default!**)

Null-Werte werden vor der Auswertung entfernt (außer im Falle COUNT(*)).

Beispiel

Anzahl von Weinen

```
SELECT COUNT(*) AS number  
FROM wine;
```

Die Anzahl von unterschiedlichen Regionen, wo Weine produziert werden

```
SELECT COUNT(DISTINCT region)  
FROM producer;
```

Die Namen und Jahre der Weine, die älter sind als der Durchschnitt

```
SELECT name, year  
FROM wine  
WHERE year < ( SELECT AVG(year) FROM wine);
```

Aggregatfunktionen in der WHERE-Klausel

Aggregatfunktionen liefern nur einen Wert

⇒ Einsatz in Konstanten- Selektionen der WHERE-Klausel möglich

Alle Weingüter, die nur einen Wein produzieren

```
SELECT * FROM producer e  
WHERE 1 = (SELECT COUNT(*) FROM wine w  
          WHERE w.vineyard = e.vineyard);
```

8 Anfragesprache (DQL) - Weiteres zu SQL

- Weitere Join-Varianten
- Aggregation
- **Gruppierung**
- Null-Werte
- Rekursion
- Anzahl von Ergebnissen begrenzen

Gruppierung

Notation

SELECT ...

FROM ...

[WHERE ...]

[**GROUP BY** attributliste]

[**HAVING** bedingung];

Menti

Frage 3

Menti

Fragen 4–6

Richtig oder Falsch?

```
SELECT COUNT(*)  
FROM course  
GROUP BY taughtBy
```

richtig

```
SELECT taughtBy, COUNT(*)  
FROM course  
GROUP BY ects
```

falsch

```
SELECT taughtBy, COUNT(*)  
FROM course  
GROUP BY ects, taughtBy
```

richtig

Die HAVING-Klausel

```
SELECT taughtBy, name, SUM(ects)
FROM course, professor
WHERE taughtBy = emplID AND rank = 'C4'
GROUP BY taughtBy, name
HAVING AVG(ects) >= 3;
```

Die HAVING-Klausel drückt eine weitere Bedingung aus, die Gruppen erfüllen müssen, um im Resultat vorzukommen.

Was wird hier berechnet? Was sind die Schritte?

8 Anfragesprache (DQL) - Weiteres zu SQL

- Weitere Join-Varianten
- Aggregation
- Gruppierung
- **Null-Werte**
- Rekursion
- Anzahl von Ergebnissen begrenzen

Null values

Manchmal sehr **überraschende Anfrageergebnisse**, wenn Null-Werte vorkommen.

```
SELECT COUNT(semester)
FROM student WHERE semester < 13 OR semester >= 13;
```

```
SELECT COUNT(semester) FROM student;
```

Beide Anfragen liefern das gleiche Ergebnis, weil Tupel mit Null-Werten in der Spalte semester nicht gezählt werden.

Auswertung bei Null-Werten

Arithmetische Ausdrücke

- Null-Werte werden „propagiert“
- $\text{null} + 1 \rightsquigarrow \text{null}$
- $\text{null} * 0 \rightsquigarrow \text{null}$

Vergleichsoperatoren

- SQL hat eine dreiwertige Logik: true, false und unknown
- Wenn mindestens eine Argument null ist, dann ist das Ergebnis unknown
- $\text{studid} = 5 \rightsquigarrow \text{unknown}$ immer, wenn studid null ist

Test, ob ein Attribut den Wert null hat

- attribute **IS** null

Auswertung bei Null-Werten

Logische Ausdrücke werden gemäß folgender Tabellen berechnet

NOT	
true	false
unknown	unknown
false	true

AND	true	unknown	false
true	true	unknown	false
unknown	unknown	unknown	false
false	false	false	false

OR	true	unknown	false
true	true	true	true
unknown	true	unknown	unknown
false	true	unknown	false

8 Anfragesprache (DQL) - Weiteres zu SQL

- Weitere Join-Varianten
- Aggregation
- Gruppierung
- Null-Werte
- **Rekursion**
- Anzahl von Ergebnissen begrenzen

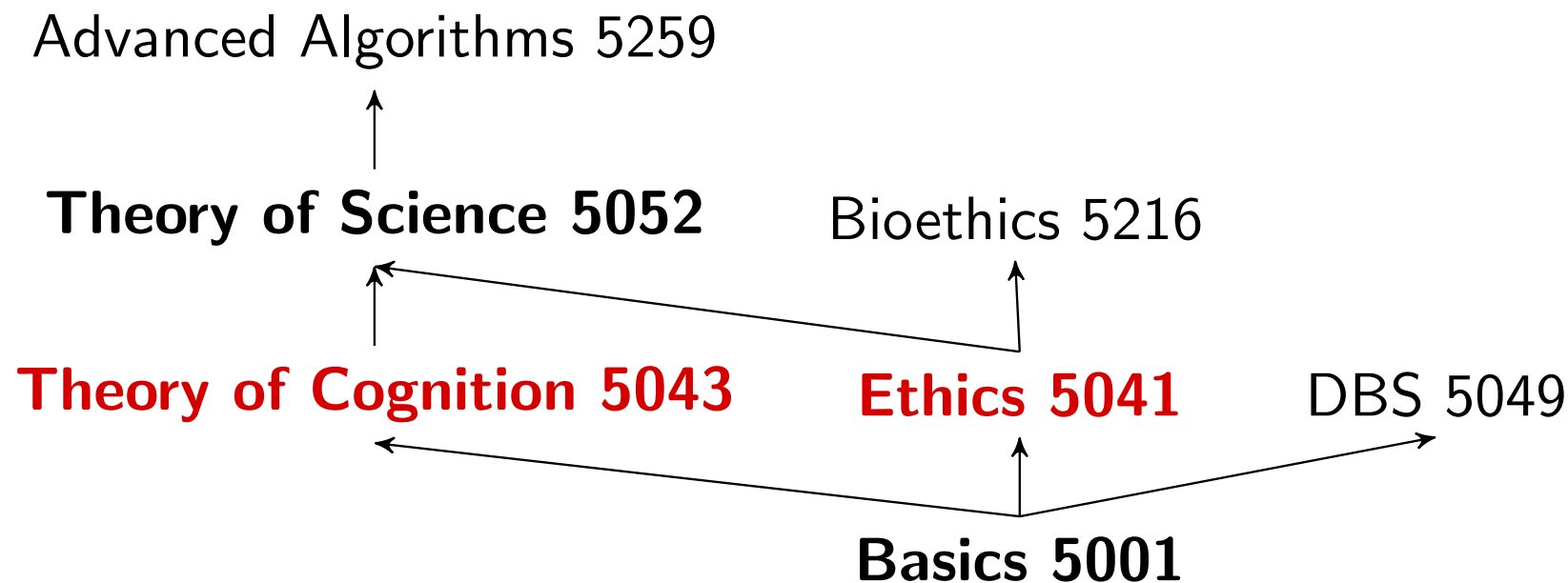
Menti

Frage 12

Berechnung der Vorgänger

```
SELECT r2.predecessor
FROM course c, requires r1, requires r2
WHERE c.title='Theory of Science' AND
      c.courseid = r1.successor AND
      r1.predecessor = r2.successor;
```

Aus welchen Vorlesungen besteht das Ergebnis dieser Anfrage?



Vorgänger der Tiefe n

```
SELECT r1.predecessor
FROM requires r1
...
requires rn_minus_1
requires rn,
course c
WHERE r1.successor= r2.predecessor AND
...
rn_minus_1.successor= rn.predecessor AND
rn.successor = c.courseid AND
c.title='Theory of Science';
```

Wie berechnet man die ganze Liste von Vorgängern beliebiger Tiefe?

Tiefe 1 UNION Tiefe 2 UNION Tiefe 3 UNION...

Rekursion in SQL

```
WITH RECURSIVE mytable(number) AS (  
    VALUES(1)  
    UNION  
    SELECT number+1  
    FROM mytable  
    WHERE number < 100  
)  
SELECT sum(number)  
FROM mytable;
```

nicht rekursiver Teil

UNION

rekursiver Teil

nur dieser darf mytable(number)
referenzieren

eigentliche Anfrage

Result: 5050 (Summe der Zahlen von 1 bis 100)

Rekursion: Anfrage für Voraussetzungen von Kursen

WITH RECURSIVE transitiveCourse (pred, succ)

AS (

SELECT predecessor, successor

FROM requires

UNION

SELECT DISTINCT t.pred, r.successor

FROM transitiveCourse t, requires r

WHERE t.succ = r.predecessor

)

SELECT *

FROM transitiveCourse

ORDER BY (pred, succ) ASC;

Rekursion: Anfrage für Voraussetzungen von Kursen

pred	succ
5001	5041
5001	5043
5001	5049
5001	5052
5001	5216
5001	5259
5041	5052
5041	5216
5041	5259
5043	5052
5043	5259
5052	5259

Advanced Algorithms 5259

Theory of Science 5052

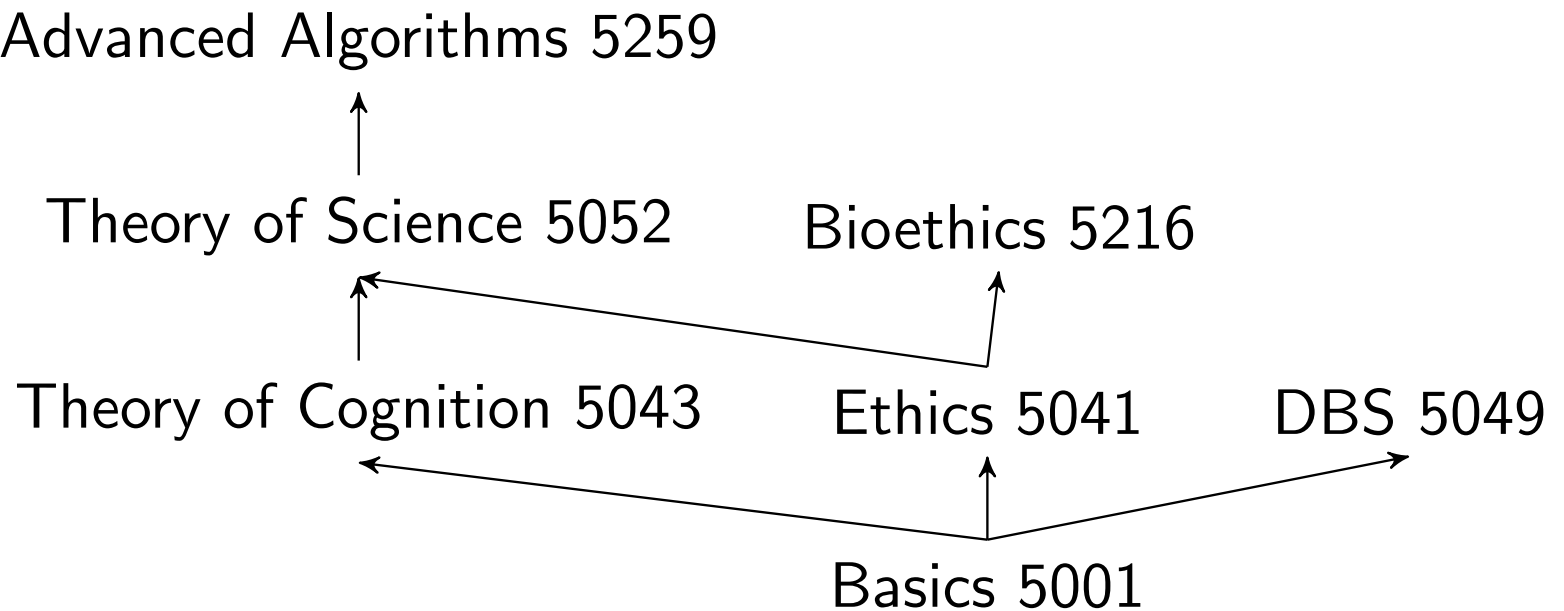
Bioethics 5216

Theory of Cognition 5043

Ethics 5041

DBS 5049

Basics 5001



Unendliche Rekursion verhindern

Unendliche Rekursion verhindern

- 1 Die meisten DBMS beschränken die Rekursionstiefe mit einem Parameter.
- 2 Direkt in der Anfrage kodieren

WITH RECURSIVE transitiveCourse (pred, succ, **depth**)

AS (

SELECT predecessor, successor, **0**

FROM requires

UNION

SELECT DISTINCT t.pred, r.successor, **t.depth+1**

FROM transitiveCourse t, requires r

WHERE t.succ = r.predecessor **AND t.depth < 1**

)

SELECT *

FROM transitiveCourse

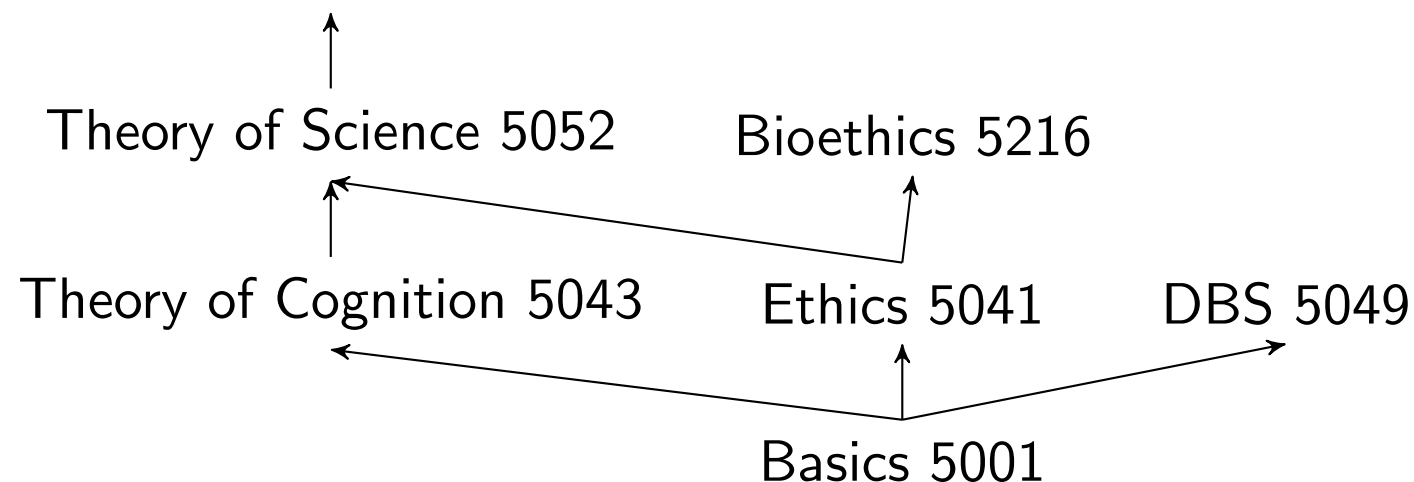
ORDER BY (pred, succ) ASC;

Unendliche Rekursion verhindern

pred	succ	depth
5001	5041	0
5001	5043	0
5001	5049	0
5001	5052	1
5001	5216	1
5001	5259	2
5041	5052	0
5041	5216	0
5041	5259	1
5043	5052	0
5043	5259	1
5052	5259	0

mit Bedingung t.depth < 1

Advanced Algorithms 5259



Alle Voraussetzungen einer bestimmten Vorlesung

WITH RECURSIVE transitiveCourse (pred, succ)

AS (

SELECT predecessor, successor

FROM requires

UNION

SELECT DISTINCT t.pred, r.successor

FROM transitiveCourse t, requires r

WHERE t.succ = r.predecessor

)

SELECT c2.title

FROM transitiveCourse tv, course c1, course c2

WHERE tv.succ = c1.courseid AND c1.title='Theory of Science'

AND tv.pred = c2.courseid;

Was genau ist das Resultat diese Anfrage?

Menti

Frage 13

Anzahl der Ergebnisse begrenzen

```
SELECT *  
FROM student  
ORDER BY semester DESC;
```

Wie kann ich das Ergebnis auf die Top-5 beschränken?

studid	name	semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

SQL 2003: Window Function

```
SELECT *  
FROM (SELECT  
    ROW_NUMBER() OVER (ORDER BY semester DESC) AS number,  
                                studid, name, semester  
FROM student  
    ) AS studTmp  
WHERE number <= 5;
```

```
SELECT *  
FROM (SELECT  
    RANK() OVER (ORDER BY semester DESC) AS number,  
                                studid, name, semester  
FROM student  
    ) AS studTmp  
WHERE number <= 5;
```

SQL 2008: Fetch-First-Klausel

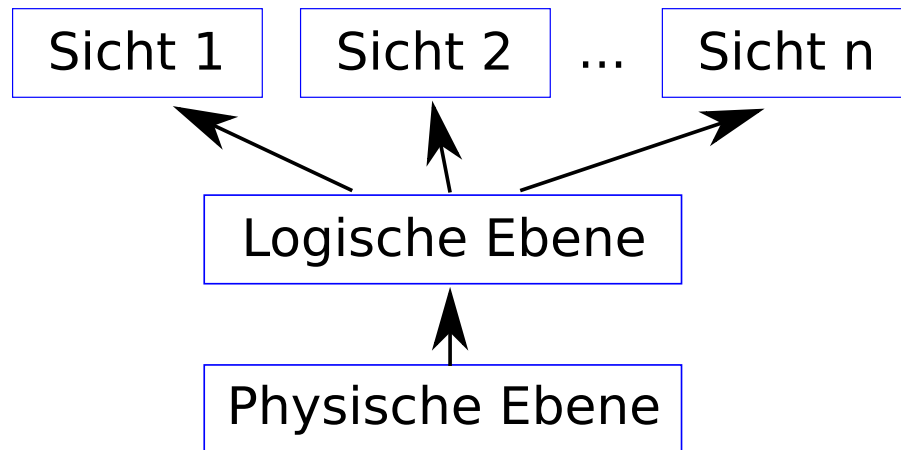
```
SELECT *  
FROM student  
ORDER BY semester DESC  
FETCH FIRST 5 ROWS ONLY;
```

Unterstützt von IBM DB2, Sybase SQL Anywhere, PostgreSQL,...

- 9 Views
 - Views erstellen und ändern
 - Materialized Views

Abstraktionsebenen eines Datenbanksystems

Änderungen auf der logischen Ebene haben keine Auswirkungen auf externe Schemata und Anwendungsprogramme.



Teilmenge des Datenbankschemas
(verschiedene Rollen)

Datenbankschema (Menge von Relationen)

Geräte, Dateien, Dateiformate (Dateien,
Festplatten)

Datenunabhängigkeit:

- Physische Unabhängigkeit: Änderungen auf der physischen Ebene haben keinen Einfluss auf die logische Ebene
- Logische Datenunabhängigkeit: Änderungen auf der logischen Ebene haben keinen Einfluss auf die Sichten (Views)

Beispiele für Views

```
CREATE VIEW profsAndtheirCourses AS
  SELECT c.title, p.name
  FROM professor p, course c
  WHERE p.empid = c.taughtBy;
```

```
SELECT * FROM profsAndtheirCourses;
```

```
CREATE VIEW ectsPerStud AS
  SELECT s.name, t.studid, SUM(c.ects) AS sum
  FROM student s, takes t, course c
  WHERE t.courseid = c.courseid AND s.studid = t.studid
  GROUP BY s.name, t.studid;
```

```
SELECT sum FROM ectsPerStud;
```

Views/Sichten können verwendet werden, um abgeleitete Attribute darzustellen (ER Diagramm).

9 Views

- Views erstellen und ändern
- Materialized Views

Menti

Fragen 14-15

Sichten vs. materialisierte Sichten

(Dynamische) Sicht

- Entspricht einem Makro für eine Anfrage
- Ergebnis der Anfrage wird nicht vorberechnet, sondern erst dann, wenn die Sicht benutzt wird

Materialisierte Sicht

- Ergebnis der Sicht wird vorberechnet
- Rechenaufwand bevor irgendeine Anfrage ausgeführt wird

Was ist die bessere Wahl?
Laufzeit vs. Updates

Updates und Views

```
CREATE VIEW howTough AS  
  SELECT empid, AVG(grade) AS avgGrade  
  FROM grades  
  GROUP BY empid;
```

```
UPDATE howTough  
  SET avgGrade = 1.0  
  WHERE empid = (SELECT empid  
                 FROM professor  
                 WHERE name = 'Socrates');
```

Was ist das Problem?

Wie sollen die Noten in der Ursprungstabelle verändert werden?

Updates und Views

```
CREATE VIEW courseView AS  
  SELECT title, ects, name  
  FROM course, professor  
  WHERE taughtBy = empid;
```

```
INSERT INTO courseView  
  VALUES ('Nihilism', '2', 'Nobody');
```

course:
{ [courseid, title, ects, taughtBy] }

professor:
{ [empid, name, rank, office] }

Was ist das Problem?

Welche Tupel sollen in die Ursprungstabellen eingefügt werden?

Änderbarkeit von Sichten in SQL

Daten in einer View sind veränderbar (update/insert/delete), wenn ...

in SQL-92

- nur eine Tabelle
- nur Selektion und Projektion
- keine Aggregatfunktionen, Gruppierung und Duplikateliminierung

in SQL-99

- wie SQL-92
- mehrere Tabellen und Attribute, wenn diese mit Hilfe des Primärschlüssels eindeutig zugeordnet werden können
- D.h. auch Views mit Joins können manchmal geändert werden

10 Integritätsbedingungen

- Statische Integritätsbedingungen
- Dynamische Integritätsbedingungen
- Komplexe Konsistenzbedingungen

Integritätsbedingungen

- Zusätzliches Instrument, um Inkonsistenten zu verhindern
- Ziel: das Einfügen inkonsistenter Daten verhindern

Welche Integritätsbedingungen haben wir in der Vorlesung schon kennengelernt?

- Keine zwei Tupel haben dieselben Werte in den Schlüsselattributen
- Kardinalitäten/Stelligkeit von Beziehungstypen
- Generalisierung: jedes Entity vom Untertyp muss im Obertyp enthalten sein
- Domänen (d.h. zulässige Werte) von Attributen
- Primärschlüssel und Fremdschlüssel
- NOT NULL, UNIQUE

Statische Integritätsbedingungen

Statische Integritätsbedingungen müssen von jedem Zustand der Datenbank erfüllt werden.

```
CREATE TABLE professor ...  
... empid integer NOT NULL...
```

Einschränkungen des Wertebereichs

```
CREATE TABLE student ...  
... CHECK semester BETWEEN 1 AND 20...
```

Aufzählungstypen

```
CREATE TABLE professor ...  
... CHECK rank IN ('C2','C3','C4')...
```

10 Integritätsbedingungen

- Statische Integritätsbedingungen
- **Dynamische Integritätsbedingungen**
- Komplexe Konsistenzbedingungen

Referentielle Integrität

Referentielle Integrität bedeutet

Fremdschlüssel müssen auf existierende Tupel verweisen oder einen Nullwert enthalten.

Was passiert, wenn kein:e Professor:in mit empid 007 existiert?

insert into course

values (5100, 'Spying for Dummies', 4, 007);

Wie kann dieses „insert“ verhindert werden?

Festlegung von Schlüsseln

Kandidatenschlüssel

- UNIQUE
- Mehrere Kandidatenschlüssel für eine Relation sind möglich
- Darf null sein!

Primärschlüssel

- PRIMARY KEY
- Nur ein Primärschlüssel pro Relation
- Impliziert **UNIQUE NOT NULL**

Fremdschlüssel

- FOREIGN KEY
- null ist möglich, kann aber mit NOT NULL verhindert werden

Verhalten bei Änderungsoperationen

Dynamische Integritätsbedingungen müssen von Zustandsänderungen erfüllt werden.

Bei Änderung von referenzierten Daten haben wir mindestens 3 Optionen

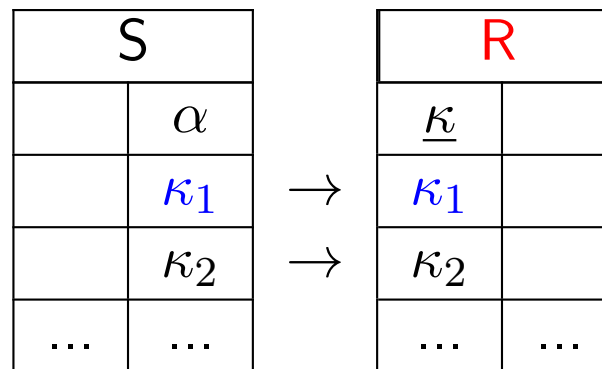
- Zurückweisen der Änderungsoperation (Default-Verhalten)
- Propagieren der Änderungen (CASCADE)
- Verweise auf „unbekannt“ setzen: **set null**

In Postgres außerdem

- Auf Defaultwert setzen (SET DEFAULT)

Beispiel

Update on table **R**



UPDATE **R**
 SET $\kappa = \kappa'_1$
 WHERE $\kappa = \kappa_1$

DELETE FROM **R**
 WHERE $\kappa = \kappa_1$

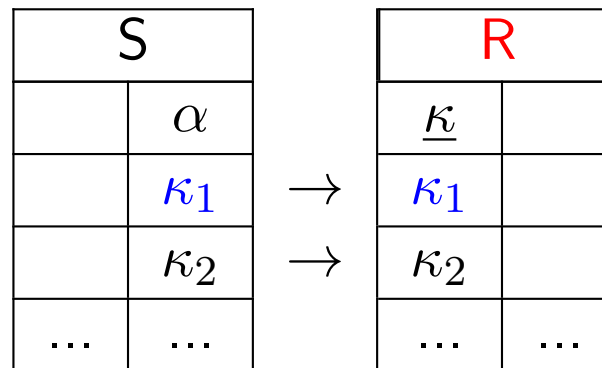
Wie soll damit umgegangen werden?

Option 1: Zurückweisen der Änderung

CREATE TABLE S

(...,

α integer REFERENCES **R**(κ));



Ergebnis der Änderungsoperation: DB bleibt unverändert

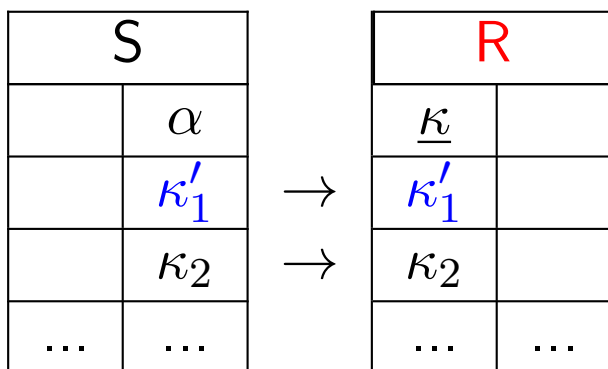
Option 2: Kaskadieren der Änderung

CREATE TABLE S

(.....,

α integer REFERENCES $R(\kappa)$

ON UPDATE **CASCADE**);



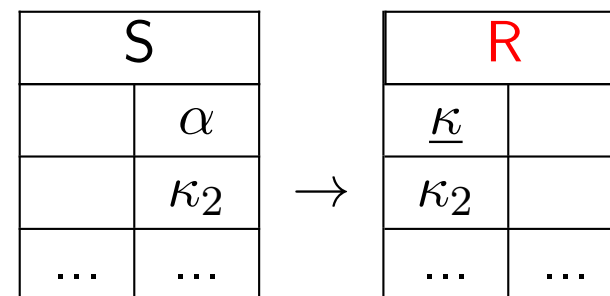
Ergebnis der Änderungsoperation:
Schlüssel in R und S geändert.

CREATE TABLE S

(.....,

α integer REFERENCES $R(\kappa)$

ON DELETE **CASCADE**);

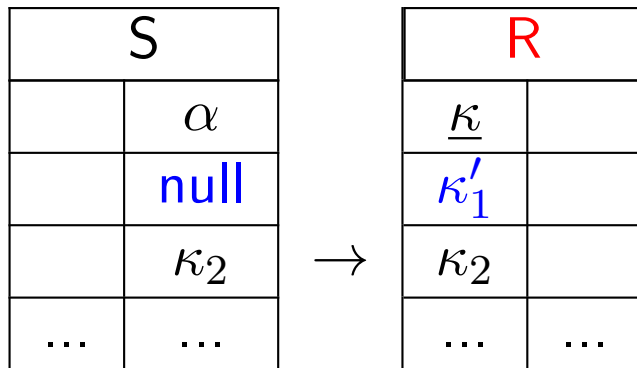


Ergebnis der Änderungsoperation:
Schlüssel in R und S gelöscht.

Option 3: Ändern und auf null setzen

CREATE TABLE S

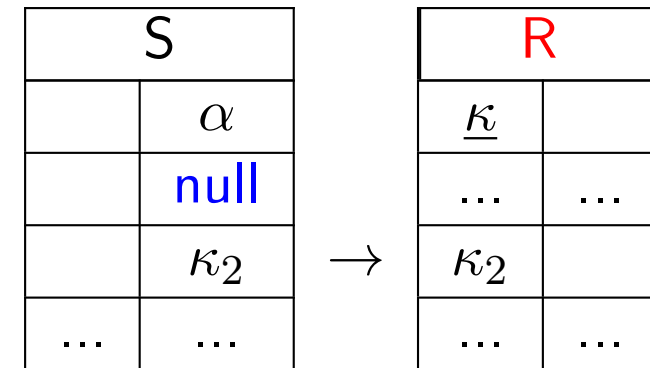
(...,
 α integer REFERENCES $R(\kappa)$
 ON UPDATE **SET NULL**);



Ergebnis der Änderungsoperation:
 Tupel aus R auf κ_1' , in S auf null
 geändert.

CREATE TABLE S

(...,
 α integer REFERENCES $R(\kappa)$
 ON DELETE **SET NULL**);



Ergebnis der Änderungsoperation:
 Tupel aus R gelöscht, Schlüssel in S auf
 null geändert.

Kaskadierendes Löschen

```
CREATE TABLE course  
(  
    ...,  
    taughtBy integer  
        REFERENCES professor(empid)  
        ON DELETE CASCADE);
```

```
CREATE TABLE takes  
(  
    ...,  
    courseid integer  
        REFERENCES course(courseid)  
        ON DELETE CASCADE);
```

```
DELETE FROM professor WHERE name = 'Socrates';
```

Kaskadierendes Löschen

professor		
empid	name	...
2125	Socrates	...
2126	Russel	...
...	...	...

course			
courseid	title	...	taughtBy
5001	Basics	...	2137
5041	Ethics	...	2125
...	...	...	...

takes	
studid	courseid
26120	5001
28106	5041
29120	5001
29120	5041
29120	5049
...	...

CREATE TABLE course

```
(...,
  taughtBy integer
    REFERENCES professor(empid)
    ON DELETE CASCADE);
```

CREATE TABLE takes

```
(...,
  courseid integer
    REFERENCES course(courseid)
    ON DELETE CASCADE);
```

DELETE FROM professor WHERE name = 'Socrates';

Was passiert?

Kaskadierendes Löschen

professor		
empid	name	...
2125	Socrates	...
2126	Russel	...
...	...	...

course			
courseid	title	...	taughtBy
5001	Basics	...	2137
5041	Ethics	...	2125
...	...	...	...

takes	
studid	courseid
26120	5001
28106	5041
29120	5001
29120	5041
29120	5049
...	...

CREATE TABLE course

```
(...,
  taughtBy integer
    REFERENCES professor(empid)
    ON DELETE CASCADE);
```

CREATE TABLE takes

```
(...,
  courseid integer
    REFERENCES course(courseid)
    ON DELETE CASCADE);
```

DELETE FROM professor WHERE name = 'Socrates';

Was passiert?

Kaskadierendes Löschen

professor		
empid	name	...
2125	Socrates	...
2126	Russel	...
...	...	...

course			
courseid	title	...	taughtBy
5001	Basics	...	2137
5041	Ethics	...	2125
...	...	...	...

takes	
studid	courseid
26120	5001
28106	5041
29120	5001
29120	5041
29120	5049
...	...

CREATE TABLE course

(...,
 taughtBy integer
 REFERENCES professor(empid)
 ON DELETE **CASCADE**);

CREATE TABLE takes

(...,
 courseid integer
 REFERENCES course(courseid)
 ON DELETE **CASCADE**);

DELETE FROM professor WHERE name = 'Socrates';

Was passiert?

10 Integritätsbedingungen

- Statische Integritätsbedingungen
- Dynamische Integritätsbedingungen
- Komplexe Konsistenzbedingungen

Komplexe Konsistenzbedingungen

```
CREATE TABLE grades (  
    studid          integer REFERENCES student ON DELETE CASCADE,  
    courseid        integer REFERENCES course,  
    grade           numeric(2,1) CHECK (grade BETWEEN 0.7 AND 5.0),  
    PRIMARY KEY(studid, courseid)  
    CONSTRAINT hasTaken  
        CHECK (EXISTS (SELECT *  
                        FROM takes h  
                        WHERE h.courseid = grades.courseid AND  
                               h.studid = grades.studid))  
);
```

- PostgreSQL unterstützt keine CHECK-Constraints, die andere Tabellen involvieren.
- Workaround durch die Verwendung von **Trigger**

Zusammenfassung

- Temporäre Tabellen (WITH)
- Null-Werte führen möglicherweise zu unerwarteten Ergebnissen
- Rekursive Anfragen
- Größe des Ergebnisses beschränken (LIMIT, FETCH FIRST n ROWS ONLY, ...)
- Views können wie „Makros“ verwendet werden
- Materialisierte Sichten für schnellere Anfragebearbeitung
- Integritätsbedingungen, um Inkonsistenzen in den Daten zu vermeiden
- CASCADE vorsichtig verwenden
- Triggers sind mächtiger als CHECK-Bedingungen aber auch „gefährlicher“

- Varianten von Triggern in Postgres 8.4

12 Anfragesprache (DQL)

- Der SFW-Block
- Mengenoperationen
- Geschachtelte Anfragen

BETWEEN und LIKE

```
SELECT name, studid  
FROM student  
WHERE semester BETWEEN 1 AND 4;
```

Wir könnten das auch mithilfe der Vergleichsoperatoren ausdrücken. Wie?

```
WHERE semester  $\geq$  1 AND semester  $\leq$  4
```

```
SELECT name, studid  
FROM student  
WHERE name LIKE 'M%';
```

„%“ steht für einen beliebigen String (inklusive der Länge 0)
„_“ steht für einen beliebigen Char.

UNION

R

A	B	C
1	2	3
2	3	4

S

A	C	D
2	3	4
2	4	5

R UNION S

A	B	C
1	2	3
2	3	4
2	4	5

R UNION ALL S

A	B	C
1	2	3
2	3	4
2	3	4
2	4	5

R UNION CORRESPONDING S

A	C
1	3
2	4
2	3

R UNION CORRESPONDING BY (A) S

A
1
2

INTERSECT und EXCEPT

A	x
	1
	2
	3

B	x
	2
	3
	4

(SELECT * FROM A)

INTERSECT

(SELECT * FROM B);

A	x
	2
	3

(SELECT * FROM A)

EXCEPT

(SELECT * FROM B);

A	x
	1

Mengenoperationen in Kombination mit ALL und CORRESPONDING werden nicht von allen DBMS unterstützt.

Auswertung von unkorrelierten geschachtelte Anfragen

Logische Schritte

- 1 Auswertung der inneren Anfrage zu den Weingütern aus Bordeaux
- 2 Einsetzen des Ergebnisses als Menge von Konstanten in die äußere Anfrage
- 3 Auswertung der modifizierten Anfrage

SELECT name

FROM wine

WHERE vineyard IN (
 'Château La Rose', 'Château La Pointe');

name
La Rose Grand Cru

Auswertung von unkorrelierten geschachtelte Anfragen

Interne Evaluationsstrategie

Umformung in einen Join

```
SELECT name  
FROM wine  
WHERE vineyard IN (SELECT vineyard  
                    FROM producer  
                    WHERE region='Bordeaux');
```

```
SELECT name  
FROM wine NATURAL JOIN producer  
WHERE region = 'Bordeaux';
```

Auswertung von korrelierten geschachtelte Anfragen

```
SELECT * FROM producer
WHERE 1999 IN (
    SELECT year FROM wine
    WHERE color='red' AND wine.vineyard = producer.vineyard);
```

Alternative

Umformulierung in einen Join

Wie?

```
SELECT vineyard, area, region
FROM producer, wine
WHERE wine.vineyard = producer.vineyard
    AND color='red' AND year=1999;
```


13 Anfragesprache (DQL) – Weiteres zu SQL

- Das CASE-Konstrukt
- Weitere Join-Varianten
- Aggregation
- Gruppierung

Das CASE-Konstrukt

Ausgabe/Berechnung eines Wertes in Abhängigkeit von der Auswertung einer Bedingung

CASE

WHEN *bedingung*₁ THEN *ausdruck*₁

...

WHEN *bedingung*_{*n*-1} THEN *ausdruck*_{*n*-1}

[ELSE *ausdruck*_{*n*}]

END

Einsatz in SELECT und WHERE-Klauseln

Das CASE-Konstrukt

Beispiel

```
SELECT studid, ( CASE WHEN grade < 1.5 THEN 'excellent'  
                    WHEN grade < 2.5 THEN 'good'  
                    WHEN grade < 3.5 THEN 'satisfactory'  
                    WHEN grade <= 4.0 THEN 'sufficient'  
                    ELSE 'insufficient' END)  
FROM grades;
```

Die **erste** qualifizierende WHEN-Klausel wird ausgeführt.

Three-way right outer join

```
SELECT p.empID, p.name, g.empID, g.grade, g.studid, s.studid, s.name  
FROM professor p RIGHT OUTER JOIN  
    (grades g RIGHT OUTER JOIN student s ON g.studid = s.studid)  
    ON p.empID = g.empID;
```

empid	name	empid	grade	studid	studid	name
⊥	⊥	⊥	⊥	⊥	24002	Xenokrates
2125	Socrates	2125	2.0	25403	25403	Jonas
⊥	⊥	⊥	⊥	⊥	26120	Fichte
⊥	⊥	⊥	⊥	⊥	26830	Aristoxenos
2137	Kant	2137	2.0	27550	27550	Schopenhauer
2126	Russel	2126	1.0	28106	28106	Carnap
⊥	⊥	⊥	⊥	⊥	29120	Theophrastos
⊥	⊥	⊥	⊥	⊥	29555	Feuerbach

Three-way left outer join

```
SELECT p.empID, p.name, g.empID, g.grade, g.studid, s.studid, s.name  
FROM professor p LEFT OUTER JOIN  
    (grades g LEFT OUTER JOIN student s ON g.studid = s.studid)  
    ON p.empID = g.empID;
```

empid	name	empid	grade	studid	studid	name
2125	Socrates	2125	2.0	25403	25403	Jonas
2126	Russel	2126	1.0	28106	28106	Carnap
2127	Kopernikus	⊥	⊥	⊥	⊥	⊥
2133	Popper	⊥	⊥	⊥	⊥	⊥
2134	Augustinus	⊥	⊥	⊥	⊥	⊥
2136	Curie	⊥	⊥	⊥	⊥	⊥
2137	Kant	2137	2.0	27550	27550	Schopenhauer

Full Outer Join

```
SELECT p.empID, p.name, g.empID, g.grade, g.studid, s.studid, s.name
FROM professor p FULL OUTER JOIN
    (grades g FULL OUTER JOIN student s ON g.studid = s.studid)
ON p.empID = g.empID;
```

empid	name	empid	grade	studid	studid	name
2125	Socrates	2125	2.0	25403	25403	Jonas
2126	Russel	2126	1.0	28106	28106	Carnap
2127	Kopernikus	⊥	⊥	⊥	⊥	⊥
2133	Popper	⊥	⊥	⊥	⊥	⊥
2134	Augustinus	⊥	⊥	⊥	⊥	⊥
2136	Curie	⊥	⊥	⊥	⊥	⊥
2137	Kant	2137	2.0	27550	27550	Schopenhauer
⊥	⊥	⊥	⊥	⊥	26830	Aristoxenos
⊥	⊥	⊥	⊥	⊥	29120	Theophrastos
⊥	⊥	⊥	⊥	⊥	29555	Feuerbach
⊥	⊥	⊥	⊥	⊥	24002	Xenokrates
⊥	⊥	⊥	⊥	⊥	26120	Fichte

Aggregatfunktionen

Argumente einer Aggregatfunktion

- ein Attribut der durch die FROM-Klausel spezifizierten Relation
- ein gültiger skalarer Ausdruck oder im Falle der COUNT-Funktion auch das Symbol *
- Aggregatfunktionen dürfen nicht geschachtelt werden!

Beispiel

```
SELECT AVG(semester)  
FROM student;
```

Beispiele

Anzahl von Weinen

```
SELECT COUNT(*) AS number  
FROM wine;
```

Die Anzahl von unterschiedlichen Regionen, die Wein produzieren

```
SELECT COUNT(DISTINCT region)  
FROM producer;
```

Die Namen und Jahre der Weine, die älter sind als der Durchschnitt

```
SELECT name, year  
FROM wine  
WHERE year < ( SELECT AVG(year) FROM wine);
```


Schachtelung von Aggregatfunktionen

Aggregatfunktionen dürfen nicht geschachtelt werden!

FALSCH

```
SELECT  $f_1(f_2(A))$  AS result  
FROM  $R \dots$ ;
```

Möglich

```
SELECT  $f_1(\text{temp})$  AS result  
FROM ( SELECT  $f_2(A)$  AS temp FROM  $R \dots$  );
```

Aggregatfunktionen in der WHERE-Klausel

Aggregatfunktionen produzieren einen einzigen Wert $\rightsquigarrow$ verwendbar für vergleiche mit Konstanten in der WHERE-Klausel.

Alle Weingüter, die nur einen Wein herstellen

```
SELECT * FROM producer e
WHERE 1 = (SELECT COUNT(*) FROM wine w
          WHERE w.vineyard = e.vineyard);
```

- 14 Anfragesprache (DQL) – Noch mehr SQL
 - Gruppierung in geschachtelten Anfragen
 - Window Funktionen

Aggregation in geschachtelten Anfragen

```
SELECT tmp.studid, tmp.name, tmp.courseCount
FROM (SELECT s.studid, s.name, COUNT(*) AS courseCount
      FROM student s, takes h
      WHERE s.studid = h.studid
      GROUP BY s.studid, s.name) tmp
WHERE tmp.courseCount > 2;
```

Ergebnis der inneren Anfrage

studid	name	courseCount
28106	Carnap	4
29120	Theophrastos	3
29555	Feuerbach	2
27550	Schopenhauer	2
25403	Jonas	1
26120	Fichte	1

Gesamtergebnis

studid	name	courseCount
28106	Carnap	4
29120	Theophrastos	3

Decision-Support mit geschachtelten Unteranfragen

```
SELECT h.courseid, h.nmbPerCourse, g.totalNumber,  
h.nmbPerCourse / CAST(g.totalNumber AS float) AS share  
FROM (SELECT courseid, COUNT(*) AS nmbPerCourse  
      FROM takes  
      GROUP BY courseid) h,  
(SELECT COUNT(*) AS totalNumber  
 FROM student) g;
```

Typumwandlung:

CAST (ausdruck AS data_type)

Was berechnet diese Anfrage?

Decision-Support mit geschachtelten Unteranfragen

```
SELECT h.courseid, h.nmbPerCourse, g.totalNumber,  
h.nmbPerCourse / CAST(g.totalNumber AS float) AS share  
FROM (SELECT courseid, COUNT(*) AS nmbPerCourse  
      FROM takes  
      GROUP BY courseid) h,  
(SELECT COUNT(*) AS totalNumber  
FROM student) g;
```

totalNumber

8

Decision-Support mit geschachtelten Unteranfragen

```
SELECT h.courseid, h.nmbPerCourse, g.totalNumber,  
h.nmbPerCourse / CAST(g.totalNumber AS float) AS share  
FROM (SELECT courseid, COUNT(*) AS nmbPerCourse  
FROM takes  
GROUP BY courseid) h,  
(SELECT COUNT(*) AS totalNumber  
FROM student) g;
```

totalNumber
8

courseid	nmbPerCourse
5259	1
5022	2
5001	4
5052	1
5049	1
5216	1
4052	1
5041	2

Decision-Support mit geschachtelten Unteranfragen

```
SELECT h.courseid, h.nmbPerCourse, g.totalNumber,  
h.nmbPerCourse / CAST(g.totalNumber AS float) AS share  
FROM (SELECT courseid, COUNT(*) AS nmbPerCourse  
FROM takes  
GROUP BY courseid) h,  
(SELECT COUNT(*) AS totalNumber  
FROM student) g;
```

courseid	nmbPerCourse	totalNumber
5259	1	8
5022	2	8
5001	4	8
5052	1	8
5049	1	8
5216	1	8
4052	1	8
5041	2	8

Decision-Support mit geschachtelten Unteranfragen

```
SELECT h.courseid, h.nmbPerCourse, g.totalNumber,  
h.nmbPerCourse / CAST(g.totalNumber AS float) AS share  
FROM (SELECT courseid, COUNT(*) AS nmbPerCourse  
      FROM takes  
      GROUP BY courseid) h,  
(SELECT COUNT(*) AS totalNumber  
 FROM student) g;
```

courseid	nmbPerCourse	totalNumber	share
5259	1	8	0.125
5022	2	8	0.25
5001	4	8	0.5
5052	1	8	0.125
5049	1	8	0.125
5216	1	8	0.125
4052	1	8	0.125
5041	2	8	0.25

- 14 Anfragesprache (DQL) – Noch mehr SQL
 - Gruppierung in geschachtelten Anfragen
 - Window Funktionen

Gruppierung

Wir wollen die Namen aller Student:innen, die **Gesamtanzahl** von LVAs, die sie besuchen, und die courseids (in einer einzigen Anfrage)

GROUP BY verwenden

```
SELECT s.name, COUNT(*) AS  
courseCount, h.courseid  
FROM takes h, student s  
WHERE h.studid = s.studid  
GROUP BY s.name, h.courseid  
ORDER BY s.name ASC;
```

name	courseCount	courseid
Carnap	1	5259
Carnap	1	5216
Carnap	1	5052
Carnap	1	5041
Feuerbach	1	5001
Feuerbach	1	5022
Fichte	1	5001
Jonas	1	5022
Schopenhauer	1	5001
Schopenhauer	1	4052
Theophrastos	1	5049
Theophrastos	1	5041
Theophrastos	1	5001

Gruppierung

Wir wollen die Namen aller Student:innen, die **Gesamtanzahl** von LVAs, die sie besuchen und die courseids (in einer einzigen Anfrage).

Die Verwendung von GROUP BY führt nicht zum gesuchten Ergebnis

```
SELECT s.name, COUNT(*) AS  
courseCount, h/courseid  
FROM takes h, student s  
WHERE h.studid = s.studid  
GROUP BY s.name, h/courseid  
ORDER BY s.name ASC;
```

name	courseCount
Carnap	4
Feuerbach	2
Fichte	1
Jonas	1
Schopenhauer	2
Theophrastos	3

OVER und PARTITION BY

Wir wollen die Namen aller Student:innen, die Gesamtanzahl von LVAs, die sie besuchen und die courseids (in einer einzigen Anfrage).

```
SELECT s.name, COUNT(*) OVER  
(PARTITION BY s.studid) AS  
courseCount, h.courseid  
FROM takes h, student s  
WHERE h.studid = s.studid  
ORDER BY s.name ASC;
```

name	courseCount	courseid
Carnap	4	5259
Carnap	4	5216
Carnap	4	5052
Carnap	4	5041
Feuerbach	2	5001
Feuerbach	2	5022
Fichte	1	5001
Jonas	1	5022
Schopenhauer	2	5001
Schopenhauer	2	4052
Theophrastos	3	5049
Theophrastos	3	5041
Theophrastos	3	5001

Window Funktionen

```
SELECT s.name, COUNT(*) OVER  
(PARTITION BY s.studid) AS  
courseCount, h.courseid  
FROM takes h, student s  
WHERE h.studid = s.studid  
ORDER BY s.name ASC;
```

name	courseCount	courseid
Carnap	4	5259
Carnap	4	5216
Carnap	4	5052
Carnap	4	5041
Feuerbach	2	5001
Feuerbach	2	5022
Fichte	1	5001
Jonas	1	5022
Schopenhauer	2	5001
Schopenhauer	2	4052
Theophrastos	3	5049
Theophrastos	3	5041
Theophrastos	3	5001

Ein Window ist eine Menge von Tupeln der zugrundeliegenden Anfrage (OVER). Für jedes Ergebnistupel wird ein Wert für das Window berechnet.

PARTITION BY erlaubt es, über Spalten zu aggregieren, die nicht in einer GROUP BY-Klausel vorkommen

Window Funktionen mit mehreren PARTITION BY

```
SELECT s.name, COUNT(*) OVER (PARTITION BY s.studid) AS
courseCount,
COUNT(*) OVER (PARTITION BY h.courseid) AS studentCount, h.courseid
FROM takes h, student s
WHERE h.studid = s.studid
ORDER BY s.name ASC;
```

- **courseCount:** Anzahl der Tupel mit dem gleichen Wert studid
- **studentCount:** Anzahl der Tupel mit dem gleichen Wert courseid

name	courseCount	studentCount	courseid
Carnap	4	1	5259
Carnap	4	2	5216
Carnap	4	1	5052
Carnap	4	1	5041
Feuerbach	2	2	5001
Feuerbach	2	4	5022
Fichte	1	4	5001
Jonas	1	2	5022
Schopenhauer	2	4	5001
Schopenhauer	2	1	4052
Theophrastos	3	1	5049
Theophrastos	3	2	5041
Theophrastos	3	4	5001

Die RANK-Funktion

```
SELECT s.name, COUNT(*) OVER (PARTITION BY s.studid) AS  
courseCount,  
RANK() OVER (PARTITION BY s.name ORDER BY h.courseid) AS rank,  
h.courseid  
FROM takes h, student s  
WHERE h.studid = s.studid  
ORDER BY s.name ASC;
```

- **rank:** die Position des Tupels in der Gruppe nach Partitionierung nach s.name

name	courseCount	rank	courseid
Carnap	4	1	5041
Carnap	4	2	5052
Carnap	4	3	5216
Carnap	4	4	5259
Feuerbach	2	1	5001
Feuerbach	2	2	5022
Fichte	1	1	5001
Jonas	1	1	5022
Schopenhauer	2	1	4052
Schopenhauer	2	2	5001
Theophrastos	3	1	5001
Theophrastos	3	2	5041
Theophrastos	3	3	5049

Window Frames

```
SELECT h.studid, COUNT(h.studid) OVER ()  
FROM takes h;
```

- Die Aggregatfunktion wird über allen Tupeln evaluiert
- Keine Partitionierung
- Der gleiche Wert für alle Ergebnistupel

studid	count
26120	13
27550	13
27550	13
28106	13
28106	13
28106	13
28106	13
29120	13
29120	13
29120	13
29555	13
25403	13
29555	13

Window Frames

```
SELECT h.studid, COUNT(h.studid) OVER (ORDER  
BY h.studid)  
FROM takes h;
```

- Keine Partitionierung
- Definition von einem **sliding window**
- Das Window enthält
 - alle Tupel mit studid, die davor kommen
 - alle Tupel mit dem gleichem Wert für studid

studid	count
25403	1
26120	2
27550	4
27550	4
28106	8
28106	8
28106	8
28106	8
29120	11
29120	11
29120	11
29555	13
29555	13

Window Frames

```
SELECT studid, semester, AVG(semester) OVER  
(ORDER BY studid ASC ROWS BETWEEN 1  
PRECEDING AND 1 FOLLOWING)  
FROM student;
```

Sliding Window über gewisse Zeilen

studid	semester	avg
24002	18	15.000
25403	12	13.333
26120	10	10.000
26830	8	8.000
27550	6	5.666
28106	3	3.666
29120	2	2.333
29555	2	2.000

Regeln für Window-Funktionen

- Nur erlaubt in der SELECT (oder ORDER BY)-Klausel
- Nicht erlaubt in der GROUP BY, HAVING and WHERE-Klausel
- Kann mit den standard Gruppierungen und Aggregationen kombiniert werden

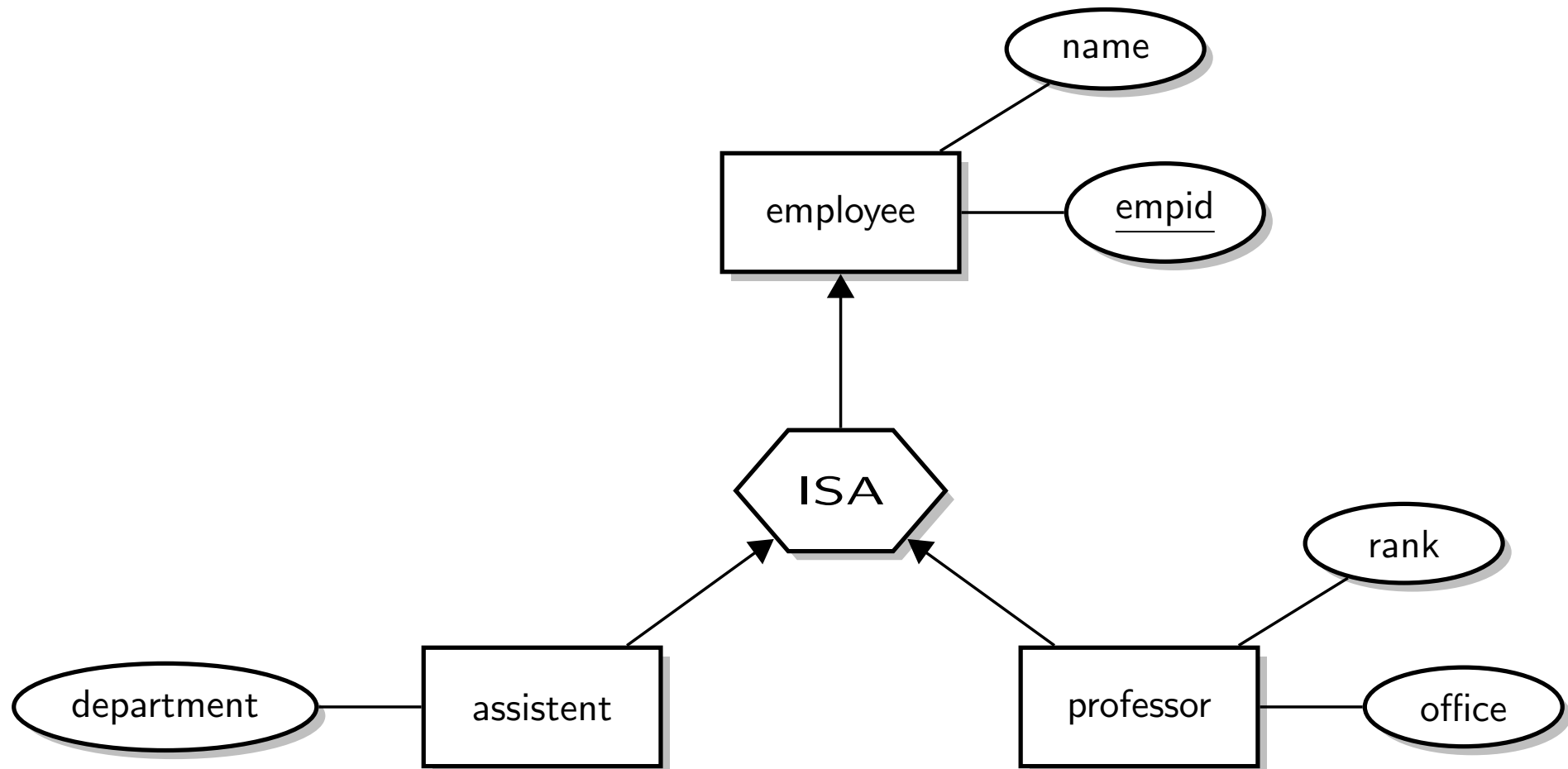
```
SELECT h.studid, COUNT(*) AS countStar, COUNT(*) OVER (ORDER BY  
h.studid) AS countPartition  
FROM takes h  
GROUP BY h.studid;
```

- **countPartition**: Sliding Window auf dem Resultat der Gruppierung!

studid	countStar	countPartition
25403	1	1
26120	1	2
27550	2	3
28106	4	4
29120	3	5
29555	2	6

- 15 Sichten
 - Generalisierung modellieren mit Sichten
 - Materialisierte Sichten

Generalisierung



Sichten um Generalisierung zu modellieren

Basis: ERDs auf Relationen abbilden durch „Partitionierung“

```
CREATE TABLE employee
  (empid      integer NOT NULL,
   name       varchar (30) NOT NULL);
```

```
CREATE TABLE profData
  (empid      integer NOT NULL,
   rank       character (2),
   office     integer);
```

```
CREATE TABLE assistantData
  (empid      integer NOT NULL,
   department  varchar (30));
```

Subtypen as Sichten

```
CREATE VIEW professor AS  
  SELECT *  
  FROM employee e JOIN profData d  
    ON e.empid = d.empid;
```

```
CREATE VIEW assistent AS  
  SELECT *  
  FROM employee e JOIN assistentData d  
    ON e.empid = d.empid;
```

Subtypen als Sichten

- Nur bei Zugriff auf professor oder Assistenten muss die View ausgeführt werden
- D.h. der Zugriff auf den Obertyp ist bevorzugt (günstig).
- Berechnung als Join

Alternative Variante

Basis: ERDs auf Relationen durch „Hauptklassen“

```
CREATE TABLE professor
```

```
    (empid      integer NOT NULL,  
     name       varchar (30) NOT NULL,  
     rank       character (2),  
     office     integer);
```

```
CREATE TABLE assistant
```

```
    (empid      integer NOT NULL,  
     name       varchar (30) NOT NULL,  
     department  varchar (30);
```

```
CREATE TABLE otherEmployees
```

```
    (empid      integer NOT NULL,  
     name       varchar (30) NOT NULL);
```

Obertyp als Sicht

```
CREATE VIEW employee AS
    (SELECT empid, name
     FROM professor)
UNION
    (SELECT empid, name
     FROM assistant)
UNION
    (SELECT *
     FROM otherEmployees);
```

Obertyp als Sicht

- Nur bei Zugriff auf Angestellte muss die View ausgeführt werden
- D.h. der Zugriff auf die Untertypen ist bevorzugt
- Berechnung als Vereinigung

Löschen einer Regel

Regel löschen

```
DROP RULE studInsert ON stud;
```

Regeln lassen sich auch auf „normale“ Tabellen anwenden

Regel auf Tabelle student

```
CREATE RULE studentInsert  
AS ON INSERT TO student  
DO ALSO  
    INSERT INTO takes  
    VALUES (NEW.studid, 5001);
```

Regeln

```
CREATE [ OR REPLACE ] RULE name  
AS ON event TO myview [WHERE condition]  
DO [ also | instead ] action ;
```

- **event** (Aktion, die auf der Sicht ausgeführt wird):
update, **insert** oder **delete**
- **also** (Aktion, die zusätzlich zur angegebenen Operation ausgeführt werden soll)
- **instead** (Aktion, die anstelle der angegebenen Operation ausgeführt werden soll)
- **action** (auszuführende Aktion):
 - **nothing**: mache nichts
 - **command**: führe eine Aktion aus
 - **command1; command2;...**: führe mehrere Aktionen aus

16 Trigger

- Definition
- Varianten von Triggern in Postgres 8.4

Trigger

Trigger sind die allgemeinste Form von Konsistenzbedingungen

ECA pattern

- **E**vent: Ereignis
- **C**ondition: Bedingung
- **A**ction: Aktion

- Unterschiedlich implementiert je DBMS

Beispiel

```
CREATE TRIGGER noDegradation
BEFORE UPDATE ON professor
FOR EACH ROW
WHEN (old.rank IS NOT NULL)
BEGIN
    IF :old.rank = 'C3' AND :new.rank = 'C2' THEN
        :new.rank := 'C3';
    END IF;
    IF :old.rank = 'C4' THEN
        :new.rank := 'C4'
    END IF;
    IF :new.rank IS NULL THEN
        :new.rank := :old.rank;
    END IF;
END
```

16 Trigger

- Definition
- Varianten von Triggern in Postgres 8.4

Varianten von Triggern in Postgres 8.4

- FOR EACH ROW vs. FOR EACH STATEMENT
- INSERT, UPDATE, OR DELETE
- BEFORE or AFTER

Potenzielle Problem

- Ausführungsreihenfolge
before statement → before row → after row → after statement
- Sichtbarkeit von Änderungen
im Falle von update-Anweisungen: ein BEFORE-Trigger sieht die Änderungen nicht, AFTER-Trigger schon
- Rekursive Aufrufe von Triggern

“It is the trigger programmer’s responsibility to avoid infinite recursion in such scenarios.”
(Postgres Dokumentation)